Agenda: Oct 20, 2025

USC Viterbi School of Engineering

1. Lecture on Benchmarking
2. Exam 1, Rubric, Q & A regarding grading.
3. Your feedback is most welcomed. Optional anonymous survey to help me adjust for the 2nd half of the semester: https://forms.gle/n9Z8q1gTypxD6cUD7

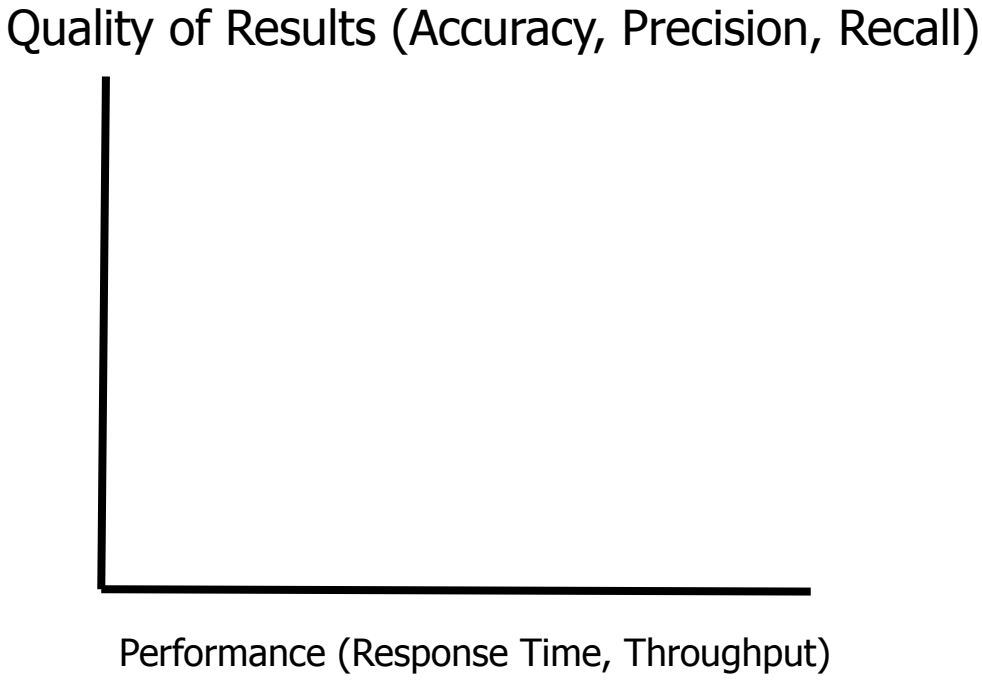
USC

USC Viterbi
School of Engineering

BG: A Benchmark to Evaluate Interactive Social Networking Actions

Sumita Barahmand & Shahram Ghandeharizadeh

Big Picture



With DBMSs, we are focused on performance because 100% accuracy/precision/recall is a must!

Motivation

- Many different DBMSs have been developed [1].

- Claims:
  - Relational join is slow. A document store such as MongoDB is faster than a SQL system by avoiding joins.
  - NoSQL systems are more effective in storing images than SQL systems.
  - NoSQL transactions (FoundationDB) are faster than SQL transactions (MySQL, Oracle, SQL Server).

[1] Rick Cattel, Scalable SQL and NoSQL Data Store, SIGMOD Record, December 2010 (Vol. 39, No. 4).

Motivation

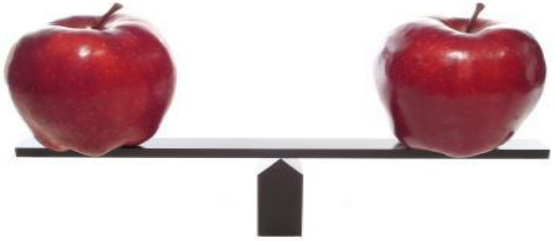
- Many different DBMSs have been developed [1].

- Which claims are true?
- How are data store vendors to identify inefficiencies in their systems and try to improve it? Benchmarks.
- Benchmarks play an important role in the application developer's decision to utilize a data store.
  - Compare various data store with one another.
  - Identify tradeoffs.
  - Understand how suitable a solution is.

[1] Rick Cattel, Scalable SQL and NoSQL Data Store, SIGMOD Record, December 2010 (Vol. 39, No. 4).

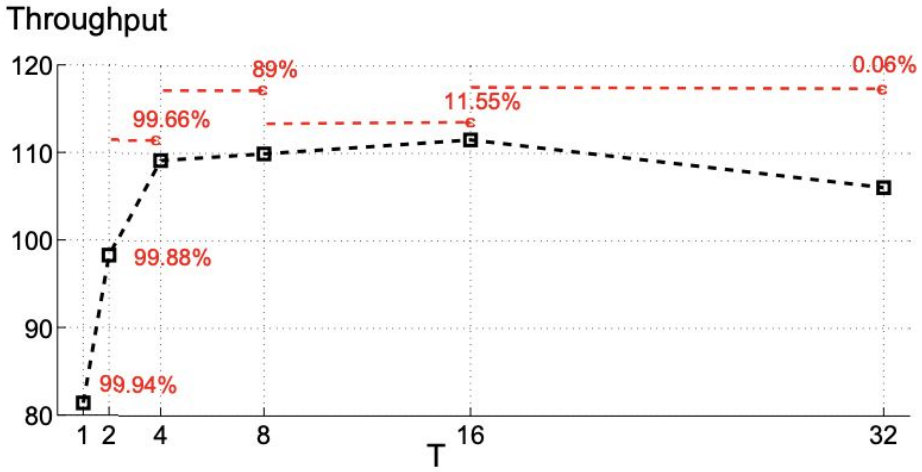
Motivation - Contd.

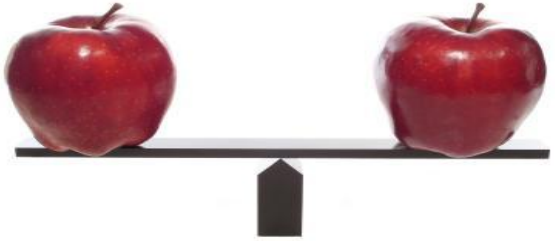
- The trade-offs between these systems are unclear. Consider the following ranking of 300+ systems: https://db-engines.com/en/ranking
- Why do systems perform as they do?
- Which components of a data store becomes the bottleneck?
- Which systems perform best for what kind of workloads?
- How accurate is the information being offered by the vendors of data stores?
- How do we compare one data store with another in a social networking application?



Throughput & Response Time

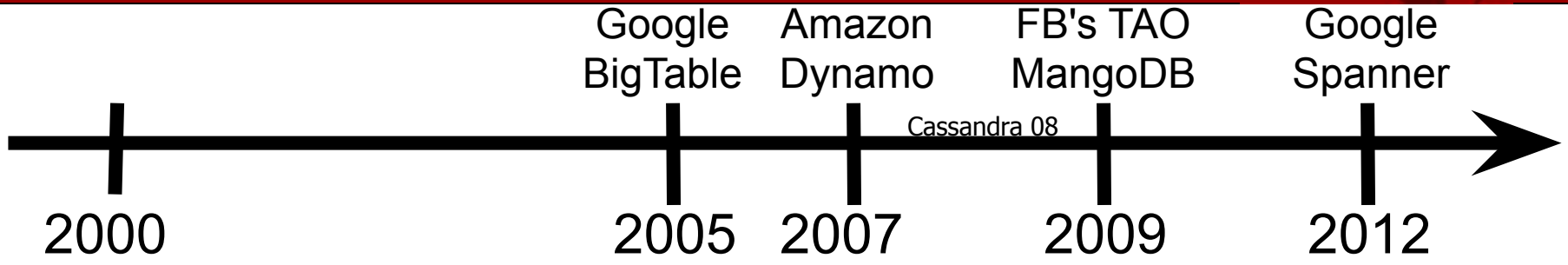
- The relationship between system response time and throughput is complex.





Questions?

No SQL Movement



What does "No SQL" mean?

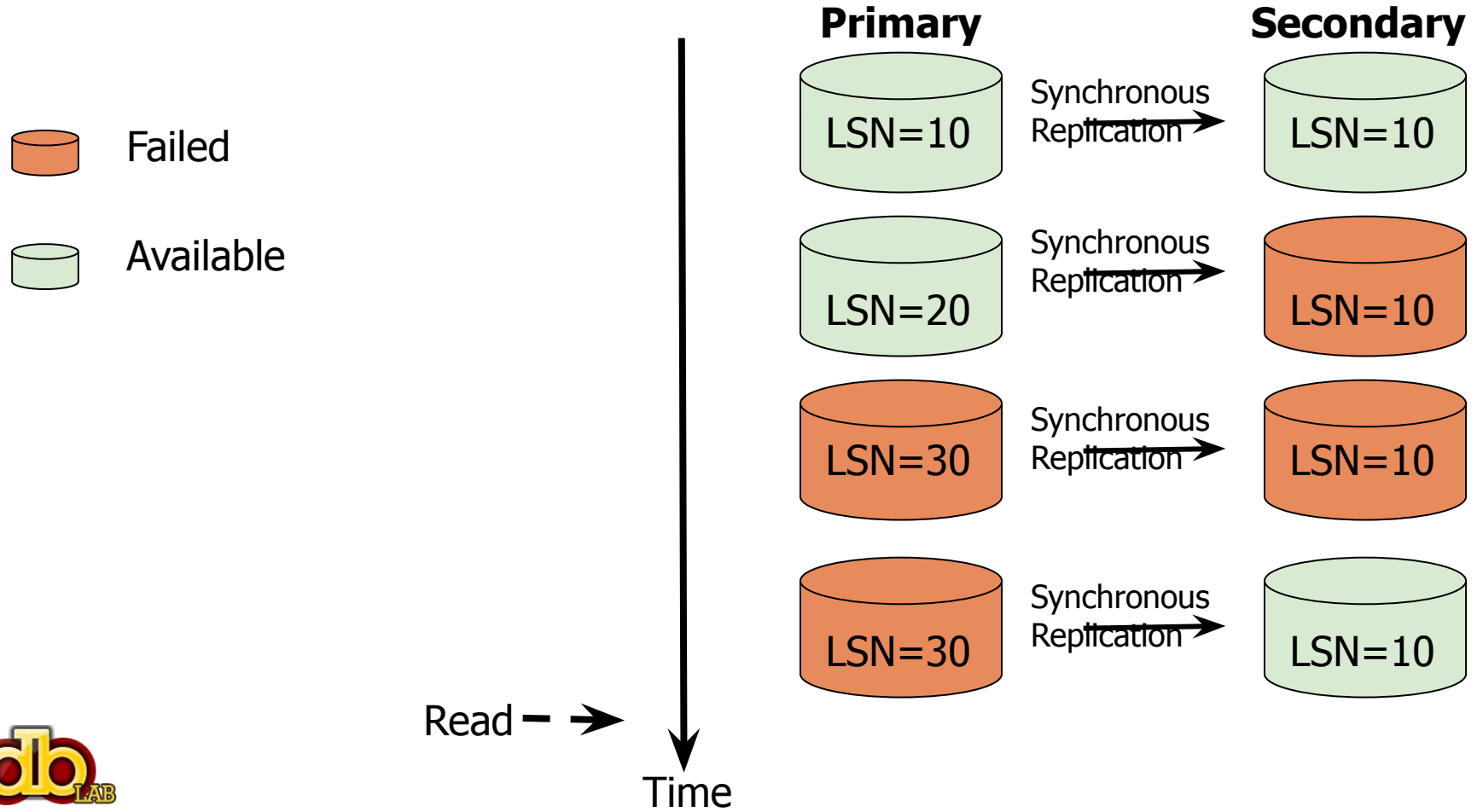
- Forget 1st normal form.
- No SQL front end. A simple call interface or protocol, e.g., FDB/DynamoDB.
- Scale.
- Replicate and/or distribute data across many servers.
- Some systems provide weaker consistency than ACID semantics. Use BASE, Basically Available, Soft state, Eventually consistent.
- Efficient use of distributed indexes and RAM for data storage, memcached.
- Ability to dynamically add new attributes to data records.

CAP Theorem

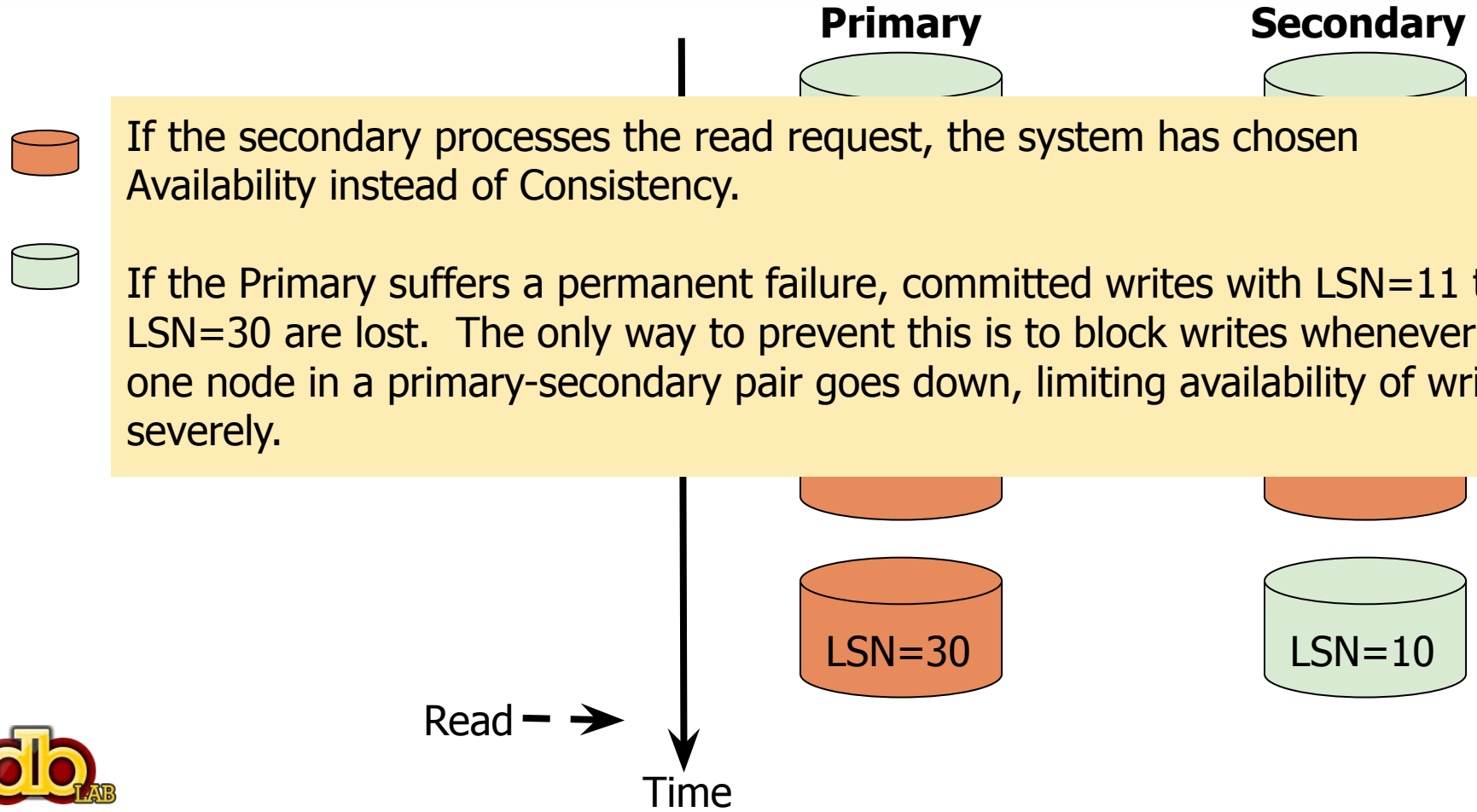
A multi-node system may provide two out of three of the following guarantees at any given time:

- **Consistency**: A read request observes the most recent write or an error.
- **Availability**: A request observes a non-error response, even when nodes have failed.
- **Partition tolerance**: The system continues to operate despite the nodes not being able to communicate with one another, e.g., node failures, loss of an arbitrary number of messages between nodes for an arbitrary duration of time.

CAP Example


Failed
Available
Primary
Secondary
LSN=10
Synchronous Replication
LSN=10
LSN=20
Synchronous Replication
LSN=10
LSN=30
Synchronous Replication
LSN=10
LSN=30
Synchronous Replication
LSN=10
Read
Time

CAP Example



If the secondary processes the read request, the system has chosen Availability instead of Consistency.

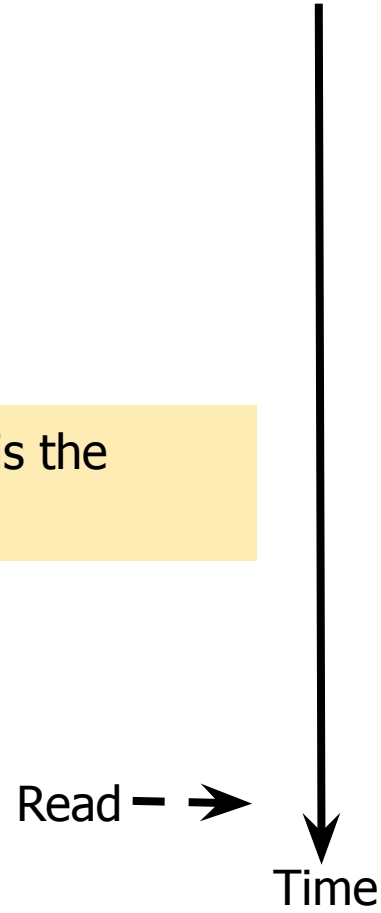
If the Primary suffers a permanent failure, committed writes with LSN=11 to LSN=30 are lost. The only way to prevent this is to block writes whenever one node in a primary-secondary pair goes down, limiting availability of writes severely.

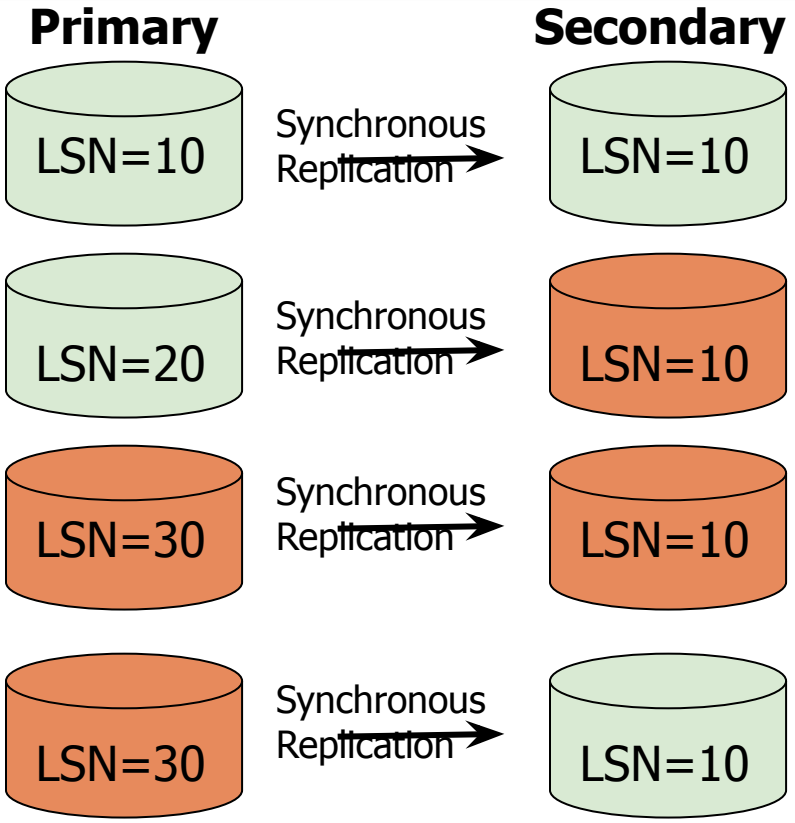
CAP Example

Failed

Available

Wait a minute, where is the network partition?





Questions?

Master the Concepts

Or, go broke!

Who needs Xacts?

Hacking, Distributed

NoSQL Meets Bitcoin and Brings Down Two Exchanges: The Story of Flexcoin and Poloniex

nosql bitcoin mongo broken

Emin Gün Sirer

April 06, 2014 at 12:15 PM

← Older

Newer →

Flexcoin was a Bitcoin exchange that shut down on March 3rd, 2014, when someone allegedly hacked in and made off with 896 BTC in the hot wallet. Because the half-million dollar heist from the hot wallet was too large for the company to bear, it folded.

I'll resist the urge to ask why they did not have deposit insurance for their hot

Example: Eventually Consistent Reads

Eventually Consistent Reads

Eventually consistent is the default read consistent model for all read operations. When issuing eventually consistent reads to a DynamoDB table or an index, the responses may not reflect the results of a recently completed write operation. If you repeat your read request after a short time, the response should eventually return the more recent item. Eventually consistent reads are supported on tables, local secondary indexes, and global secondary indexes. Also note that all reads from a DynamoDB stream are also eventually consistent.

Eventually consistent reads are half the cost of strongly consistent reads. For more information, see Amazon DynamoDB pricing.

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html

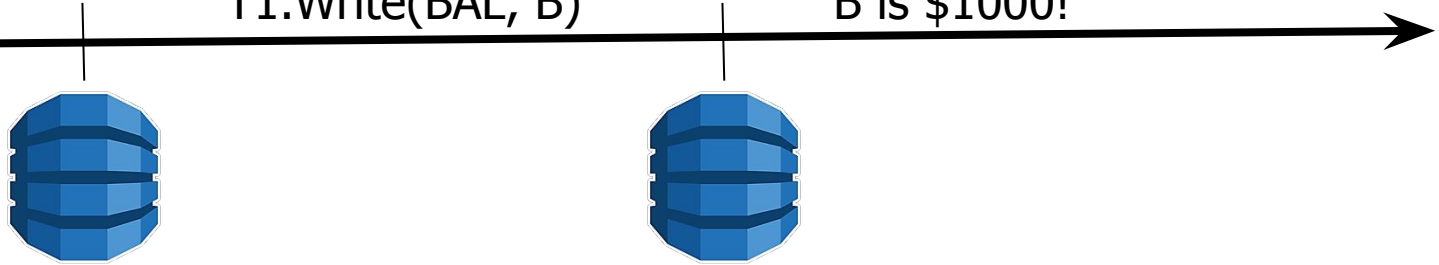
What does it mean?

- Represent accounts as rows of a DynamoDB table.
- Single threaded application.

B =T1.Read(BAL)
B = B - 700
T1.Write(BAL, B)

B =T2.Read(BAL)

B is $1000!



BAL = $1000

BAL = $1000

Example: Single-Threaded App

B =T1.Read(BAL)
B = B - 700
T1.Write(BAL, B)

B =T2.Read(BAL)
B = B - 700
T2.Write(BAL, B)

Lost $700!

BAL = $1000

BAL = $1000

BAL = $300

Example: Multi-Threaded App

B =T1.Read(BAL)
B = B - 700
T1.Write(BAL, B)

B =T2.Read(BAL)
B = B - 700
T2.Write(BAL, B)

B =T101.Read(BAL)
B = B - 700
T101.Write(BAL, B)

BAL = $1000

BAL = $1000

BAL = $300

Lost
$70,000!

Questions?

Terminology

Response time is the elapsed time from when a user issues a request to the time a response is provided to the request. There is 50th, 95th and 99th percentile. What does it mean?

- A percentile indicates the response time below which a given percentage of response times fall. For example, the 95th percentile is the response time below which 95% of the response times fall. Given n response times in an array, sort the array in ascending order, let i equal n*95/100 rounded to the nearest integer, element i of the array is the 95th percentile response time.
- 50th percentile is also known as the median.

Throughput is the rate at which a system processes requests.

Online versus offline requests:

- An online request is processed by the system as soon as it is issued, e.g., interactive user requests for stock quotes, price of goods, etc. Favors response time.
- An offline request is buffered and processed at a later time, e.g., write of information/cookie-settings gathered from a browser. Favors throughput.

BG Benchmark

USC Viterbi School of Engineering

- Compare and contrast different data stores.
- Characterize tradeoffs associated with alternative designs in a data store, e.g. Physical data design, different forms of consistency (read after write), etc.
- Choice of hardware. Ex. Different mass storage devices.
- Horizontal and vertical scalability behavior of data stores.
- Elasticity behavior of data stores.
- Quantify the behavior of a data store in the presence of various failures (either CP or AP of the CAP theorem).
- Proposed framework opens several new directions that will benefit the research community.

- Can be used by:
  - Application developers to pick the best solution for their social networking application.
  - Data store vendors to identify inefficiencies in their systems and try to improve them.

Why Are Benchmarks Important?

USC Viterbi School of Engineering

- Benchmark
  - A standard by which something can be measured or judged.
  - Quantify measures of performance.
  - Compare competing ideas, designs and implementations.
  - Identify gaps and inefficiencies in systems from which opportunities for improvements can be derived.
    - ***Engineers do a good job when performance is measurable and repeatable.***

- Designed to mimic a particular type of workload on a component/system.
  - Synthetic workload:
    - ***Used for stress testing individual components.***
  - Application workload:
    - ***Resembles the real world application workload.***
    - ***Give a better measure of how the system will work in real word.***

Challenges

- Developing benchmarks is not simple and quick!
  - Focus on metrics which are important indicators of performance or inefficiencies.
  - Should not oversimplify the typical environment.
  - Applicable to a broad spectrum of hardware/architecture.
  - Gather accurate data.
  - Results should be simple, understandable and not misinterpreted.
  - Provide horizontal scalability: The benchmark cannot be the bottleneck.
  - Vendors and users embrace it.

BG Social Benchmark

USC Viterbi School of Engineering

- Overview:
  - Data Store Agnostic
  - Complex Conceptual Schema
  - Social Actions and Social Sessions
  - Concept of SLA
  - Performance Metrics
- Scalable and Extensible Software Architecture
  - Scalable request generation: D-Zipfian Distribution
  - Stateful Request generation
- Unpredictable Data
- Freshness Confidence
- Visualization Deck
- Efficient Rating Mechanism
  - SoAR and Socialites Rating Heuristics

BG Social Actions: Simple Operations

USC Viterbi School of Engineering

Action	facebook	g+		LinkedIn	YouTube
View Profile (VP)	✓	✓	✓	✓	✓
List Friends (LF)	✓	✓	✓	✓	✓
View Friend Requests (VFR)	✓	✘	✘	✓	✘
Invite Friend (IF)	✓	Add to Circle	Follow	✓	Subscribe
Accept Friend Request (AFR)	✓	✘	✘	✓	✘
Reject Friend Request (RFR)	✓	✘	✘	✓	✘
Thaw Friendship (TF)	✓	Remove from Circle	Unfollow	✓	Unsubscribe
View Top-K Resources (VTR)	✓	✓	✓	✓	✓
View Comments on a Resource (VCR)	✓	✓	✓	✓	✓
Post Comment on a Resource (PCR)	✓	✓	Reply to a tweet	Recommend a colleague's work	Post comment on a video
Delete Comment from a Resource(DCR)	✓	✓	Delete the reply for a tweet	Withdraw recommendation	Remove comment on a video
Share Resource (SR)	✓	✓	Post a tweet	Update profile	Upload a video
View News Feed (VNF)	✓	✓	✓	✓	✓

• S. Barahmand and S. Ghandeharizadeh. **BG: A Benchmark to Evaluate Interactive Social Networking Actions**. *CIDR '13, Asilomar, CA.*

BG Architecture

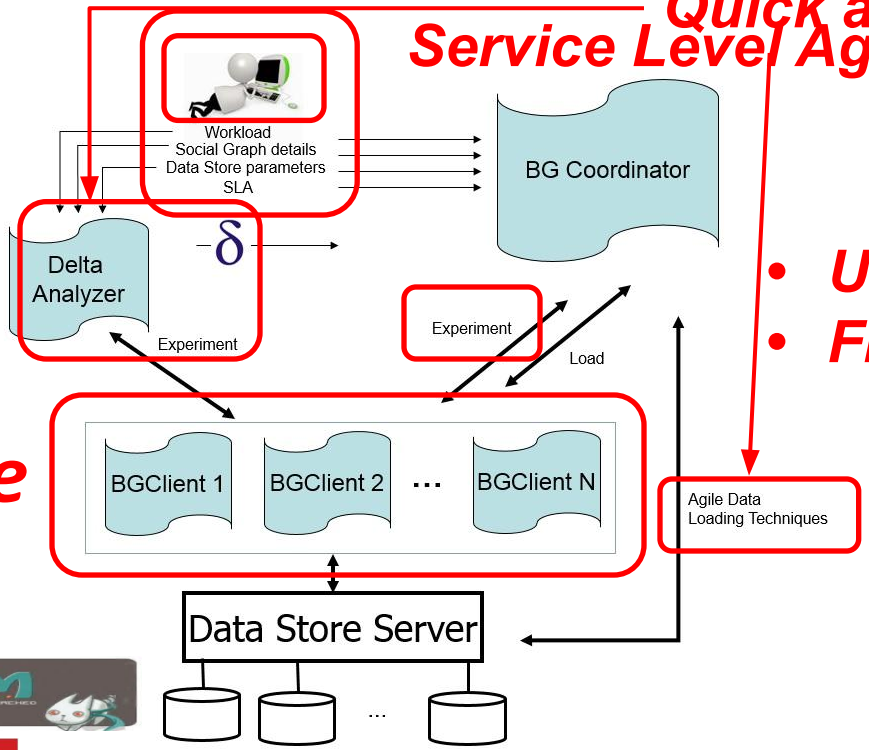
USC Viterbi School of Engineering

Visualization Tool

Emulates User Behavior

Quick and Efficient Rating

Service Level Agreement



- *Unpredictable Data*
- *Freshness Confidence*

Scalable

Neo4j

mongoDB

ORACLE

dbLAB

BG Metrics

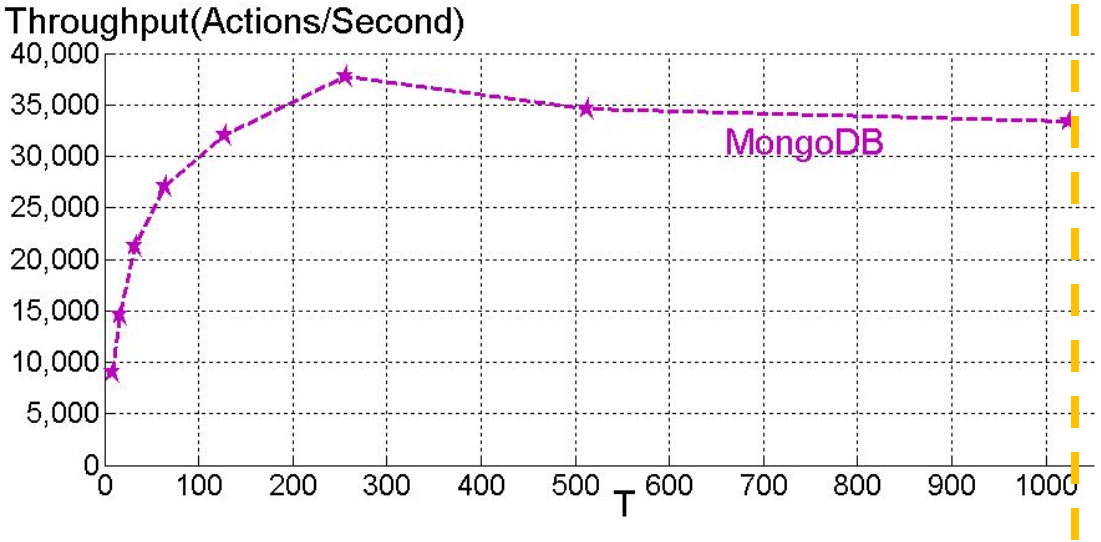
- Measured metrics:
  - Throughput.
  - Response time and confidence for a fixed response time (*90% of requests observe a response time lower than 100 msecs*).
  - Amount of unpredictable data (stale, invalid or inconsistent data).
  - Freshness confidence for a fixed duration (*95% of requests observe the freshest value in less than 10 seconds after an update occurs*).
- Rating metrics:
  - SoAR.
  - Socialites.

BG Rating Metrics

- Service Level Agreement (*SLA) Requirement:*
  - Ex. 95% of requests observe a response time equal to or faster than 100 msecs with at most 0.01% of requests observing unpredictable data for 1 hour.
- **SoAR (Social Action Rating)**: Highest number of completed actions per second satisfying an SLA requirement.
- **Socialites**: Highest number of simultaneous threads that satisfy an SLA requirement.

BG Rating Metrics - Contd.

System performance as a single number.

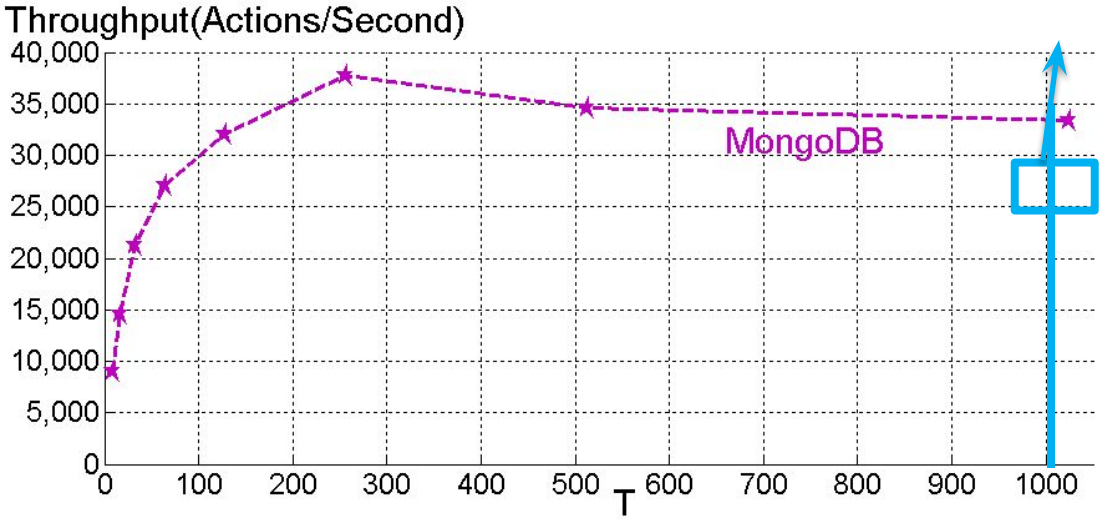


- Workload: View Profile – 10k members and no images.
- SLA: 95% of requests to observe a response time equal to or faster than 100 msecs with at most 0.01% of requests observing unpredictable data for 10 mins.
- Bottleneck: CPU of the server.

BG Rating Metrics - Contd.

USC Viterbi School of Engineering

Socialites :



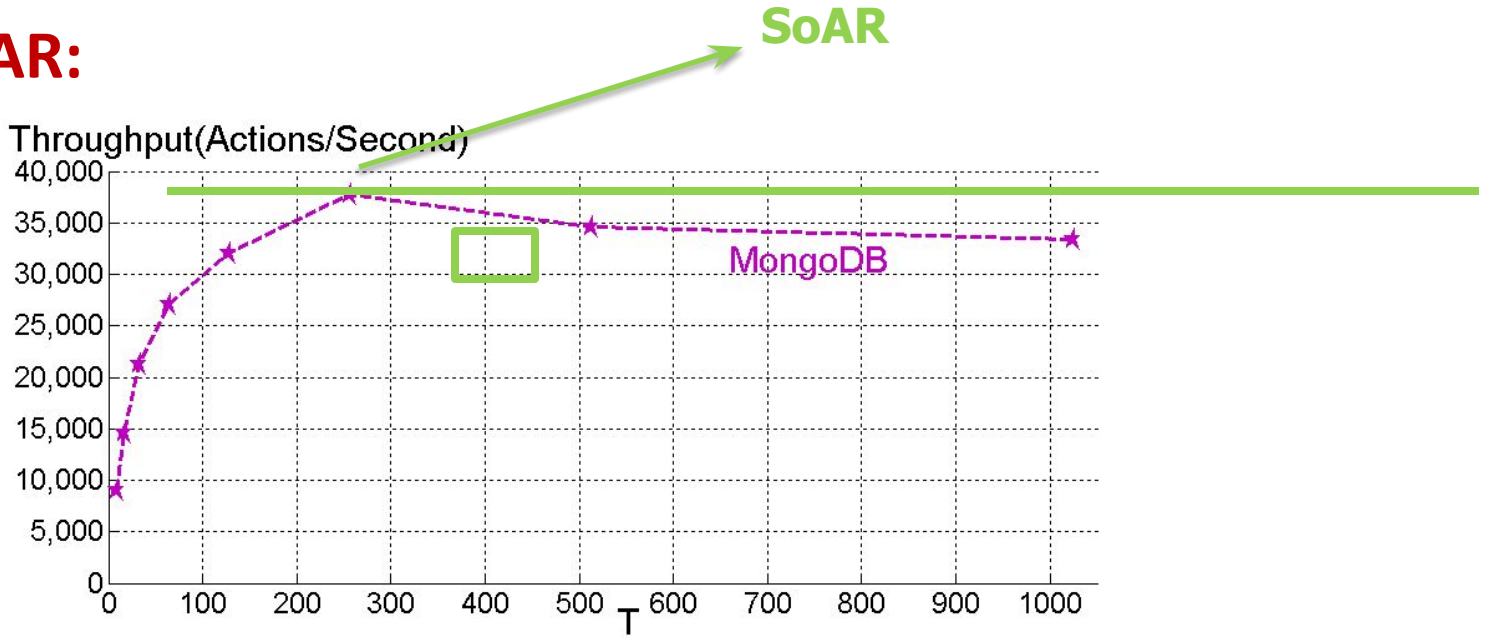
Socialites

- Workload: View Profile – 10k members and no images.
- SLA: 95% of requests to observe a response time equal to or faster than 100 msecs with at most 0.01% of requests observing unpredictable data for 10 mins.
- Bottleneck: CPU of the server.

BG Rating Metrics - Contd.

USC Viterbi School of Engineering

SoAR:

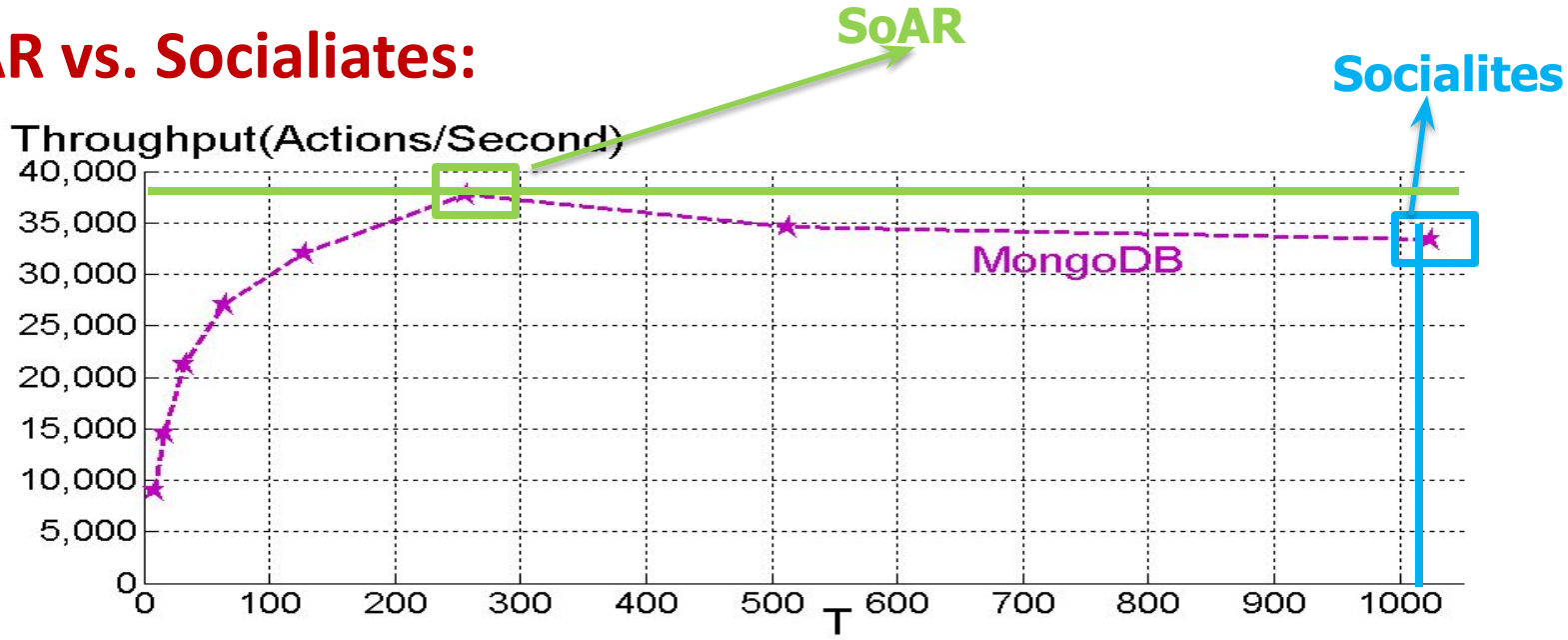


- Workload: View Profile – 10k members and no images.
- SLA: 95% of requests to observe a response time equal to or faster than 100 msecs with at most 0.01% of requests observing unpredictable data for 10 mins.
- Bottleneck: CPU of the server.

Which is more important?

USC Viterbi School of Engineering

SoAR vs. Socialites:

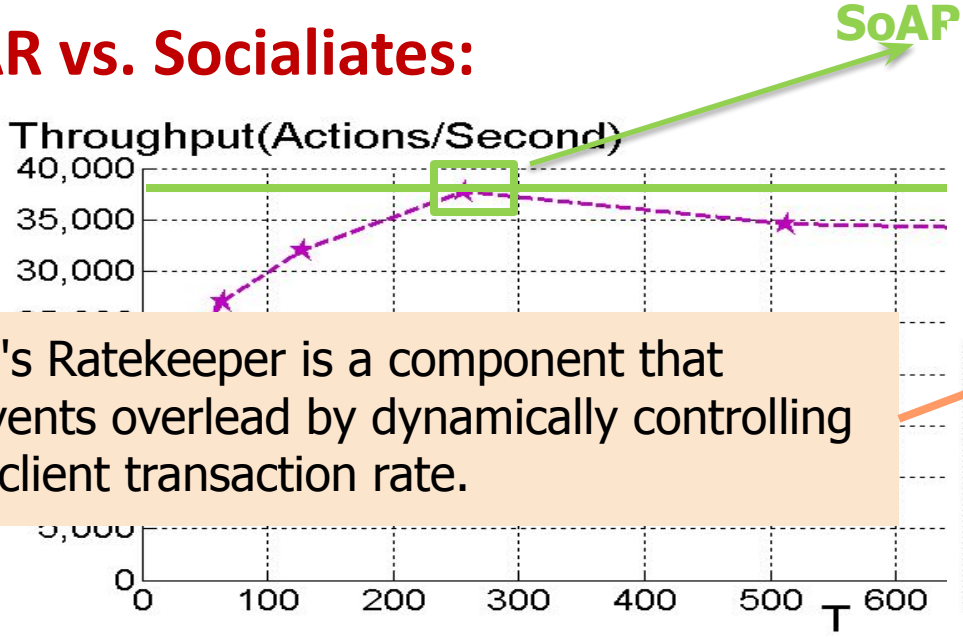


- Workload: View Profile – 10k members and no images.
- SLA: 95% of requests to observe a response time equal to or faster than 100 msecs with at most 0.01% of requests observing unpredictable data for 10 mins.
- Bottleneck: CPU of the server.

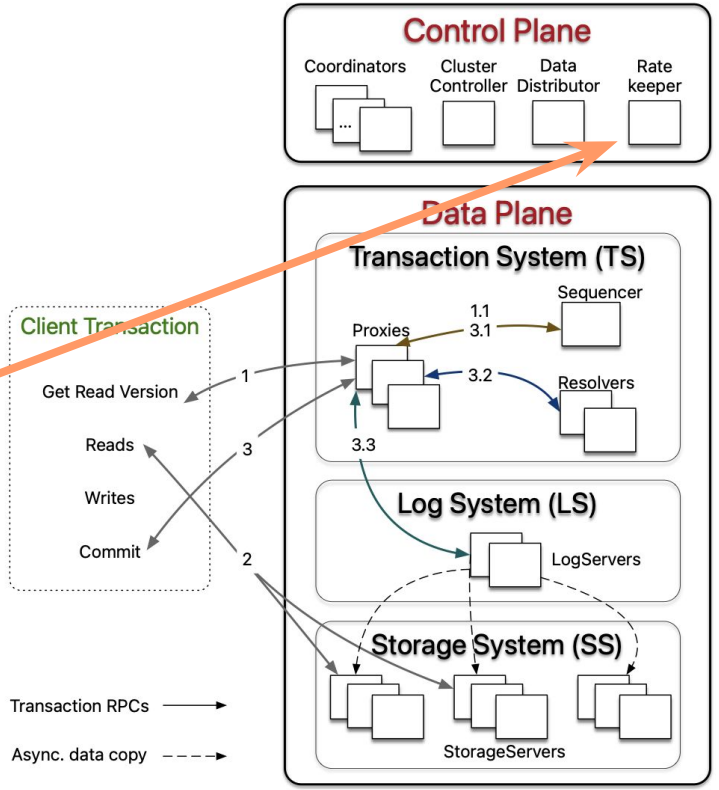
FDB Rate Keeper

USC Viterbi School of Engineering

SoAR vs. Socialiates:



FDB's Ratekeeper is a component that prevents overlead by dynamically controlling the client transaction rate.



- Workload: View Profile – 10k members and r
- SLA: 95% of requests to observe a response
  msecs with at most 0.01% of requests obse
- Bottleneck: CPU of the server.

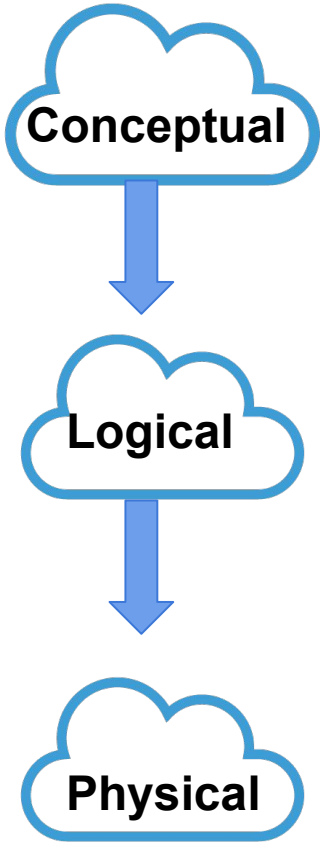
Questions?

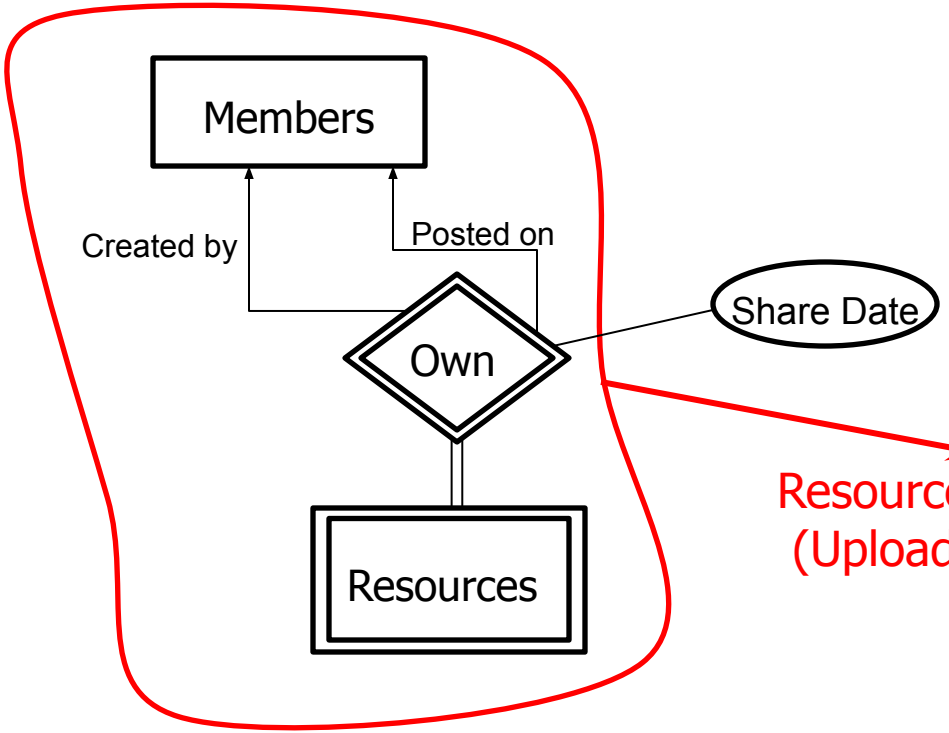
BG Workload

- Social networking workload.
  - Emulates users with:
    - Unique login sessions.
    - Different activity levels.
  - Emulates user behavior.
    - Social actions.
    - Social sessions: a sequence of actions for the same user.
    - Emulates the concept of think time and Inter-arrival time.
  - A complex conceptual schema pertaining to a social network.

BG Conceptual Schema

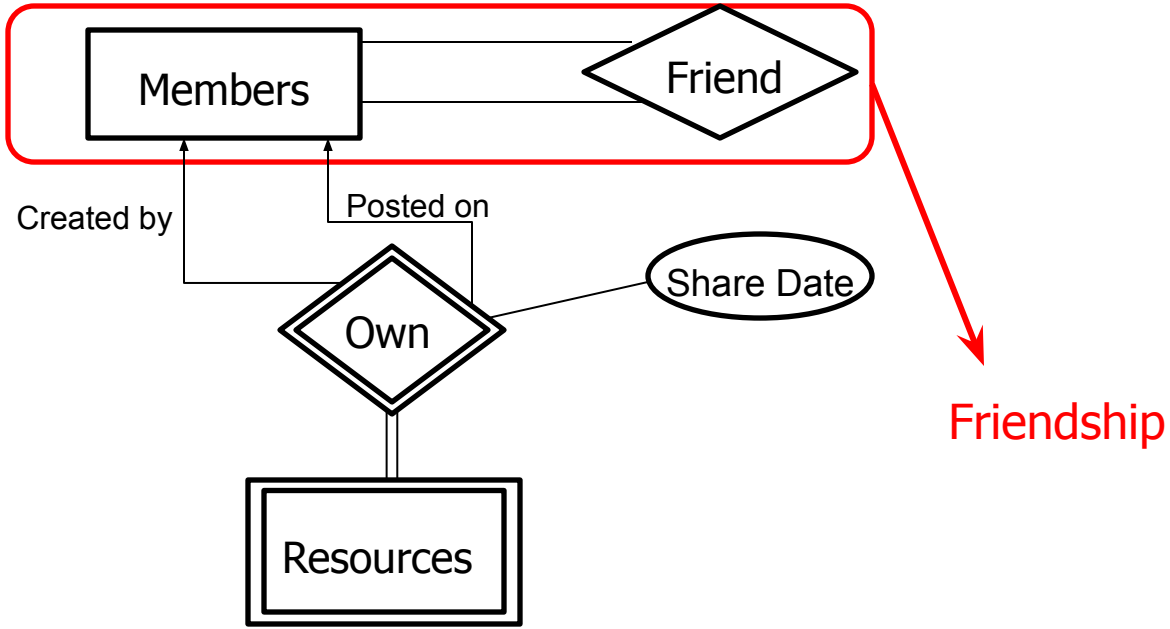
USC Viterbi School of Engineering



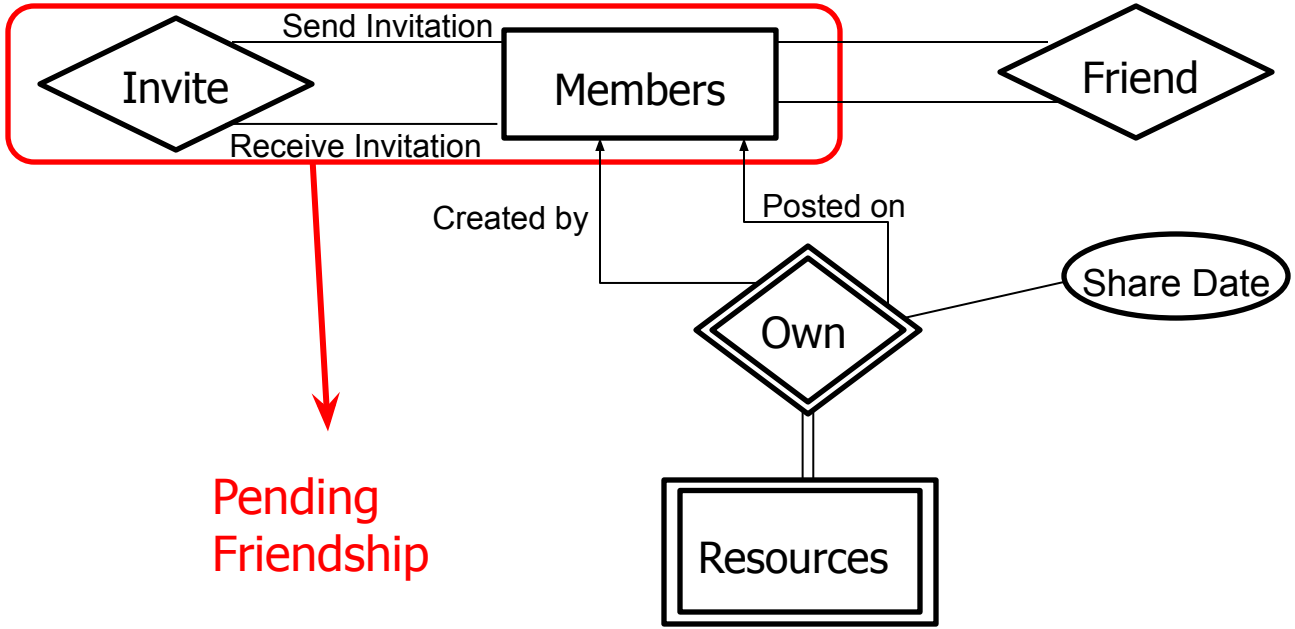


Resource Ownership
(Upload Picture, Create Status Msg

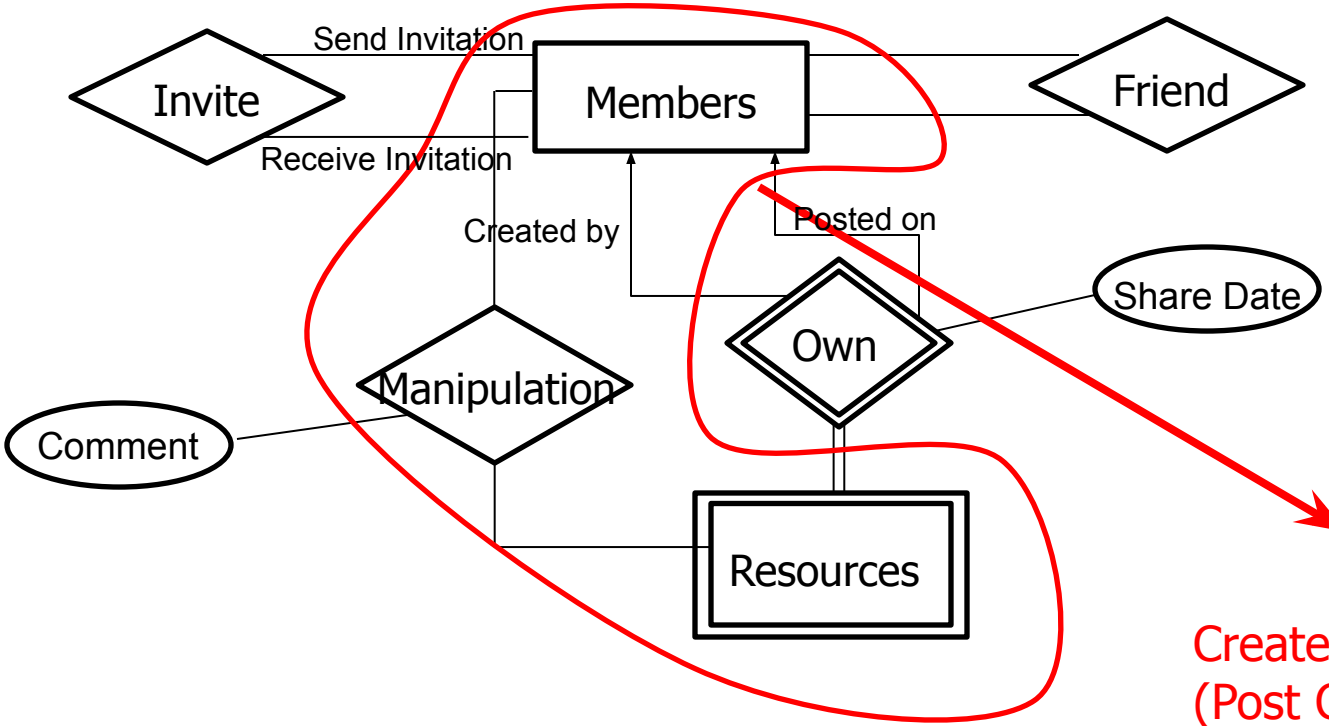
BG Conceptual Schema - Contd.



BG Conceptual Schema - Contd.

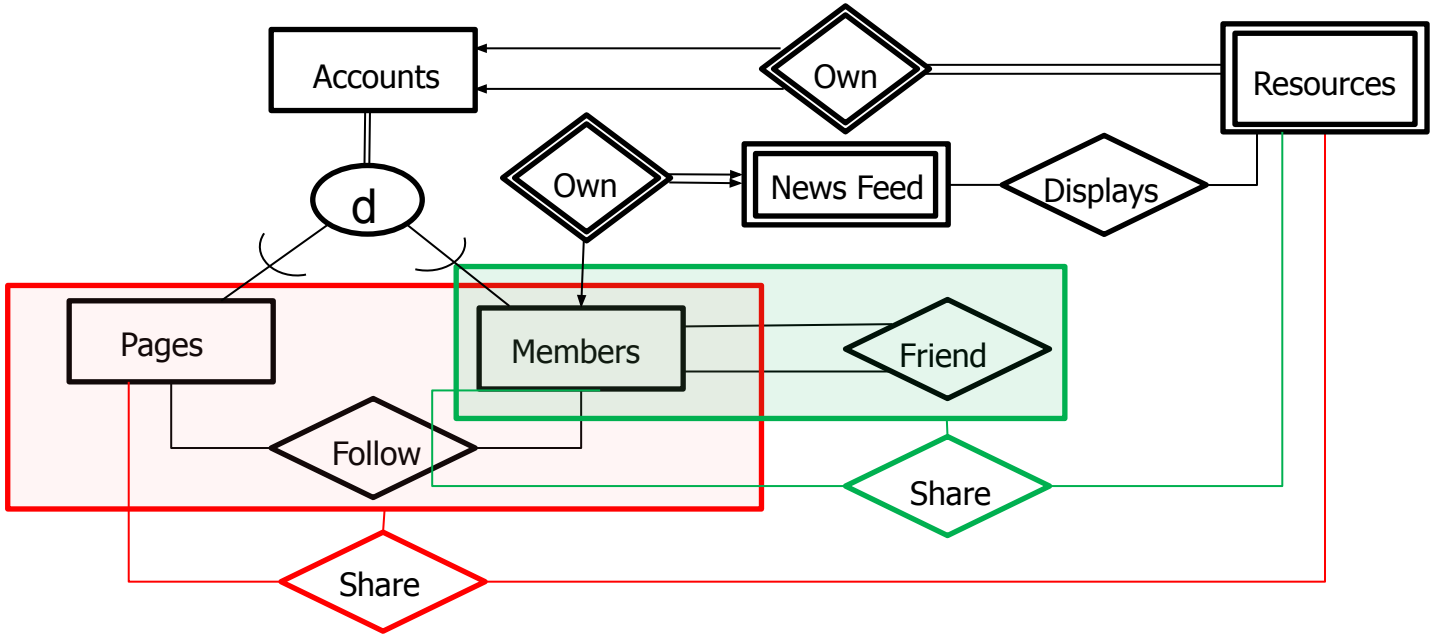


BG Conceptual Schema - Contd.



Create Content on a Resource
(Post Comment on Pic/status)

BG Conceptual Schema - Contd.



BG Social Actions

USC Viterbi School of Engineering

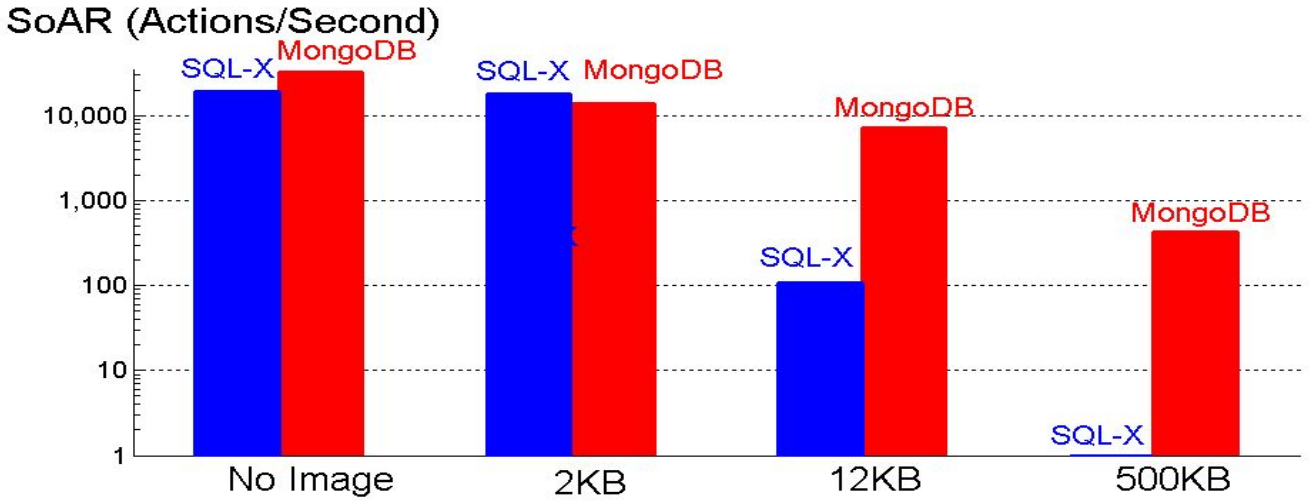
Action	facebook	g+		LinkedIn	YouTube
View Profile (VP)	✓	✓	✓	✓	✓
List Friends (LF)	✓	✓	✓	✓	✓
View Friend Requests (VFR)	✓	✘	✘	✓	✘
Invite Friend (IF)	✓	Add to Circle	Follow	✓	Subscribe
Accept Friend Request (AFR)	✓	✘	✘	✓	✘
Reject Friend Request (RFR)	✓	✘	✘	✓	✘
Thaw Friendship (TF)	✓	Remove from Circle	Unfollow	✓	Unsubscribe
View Top-K Resources (VTR)	✓	✓	✓	✓	✓
View Comments on a Resource (VCR)	✓	✓	✓	✓	✓
Post Comment on a Resource (PCR)	✓	✓	Reply to a tweet	Recommend a colleague's work	Post comment on a video
Delete Comment from a Resource(DCR)	✓	✓	Delete the reply for a tweet	Withdraw recommendation	Remove comment on a video
Share Resource (SR)	✓	✓	Post a tweet	Update profile	Upload a video
View News Feed (VNF)	✓	✓	✓	✓	✓

dbLAB

• S. Barahmand and S. Ghandeharizadeh. **BG: A Benchmark to Evaluate Interactive Social Networking Actions**. *CIDR '13, Asilomar, CA.*

Results – View Profile Action

- View Profile Action



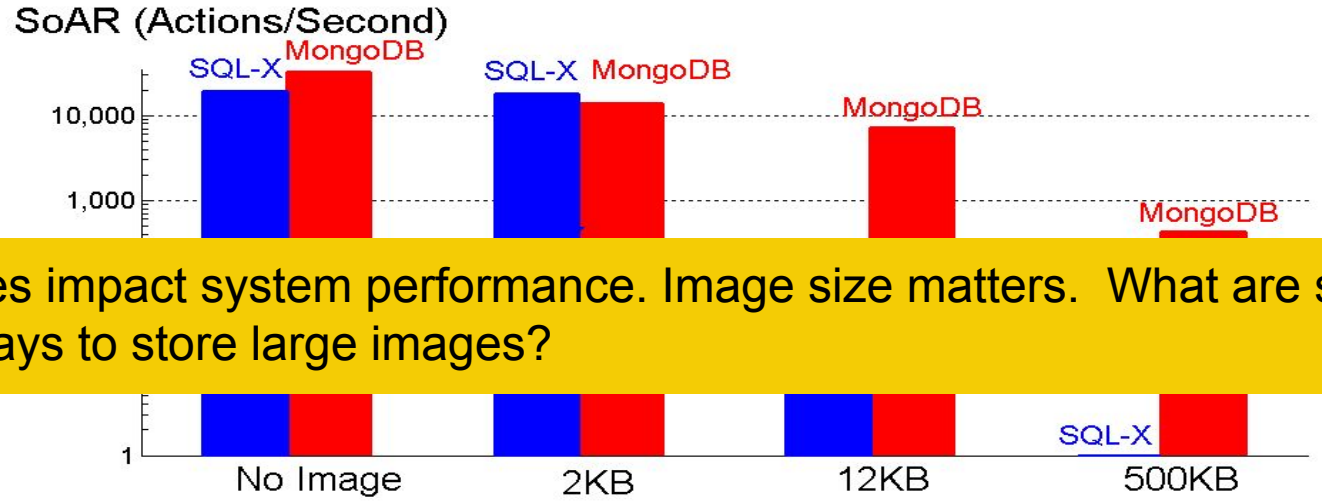
- Workload: 10k members and 100 fpm.
- With larger images the server network is the bottleneck for MongoDB. For SQL-X there are disk I/Os and the CPU of the server becomes the bottleneck.
- SLA: 95% of actions to observe a response time below 100 msecs.

• S. Barahmand and S. Ghandeharizadeh. **BG: A Benchmark to Evaluate Interactive Social Networking Actions**. *CIDR '13, Asilomar, CA.*

Results – View Profile Action

USC Viterbi School of Engineering

- View Profile Action



Images impact system performance. Image size matters. What are some of the ways to store large images?

- Workload: 10k members and 100 fpm.
- With larger images the server network is the bottleneck for MongoDB. For SQL-X there are disk I/Os and the CPU of the server becomes the bottleneck.
- SLA: 95% of actions to observe a response time below 100 msecs.

- S. Barahmand and S. Ghandeharizadeh. **BG: A Benchmark to Evaluate Interactive Social Networking Actions**. *CIDR '13, Asilomar, CA.*

List Friends Action

- List friends Action
  - Retrieves the list of friends for a member.
  - For every friend
    - Retrieve friend's basic information such as first and last name, DoB.
    - Friend's thumbnail image.

List Friends Action – Contd.

- **SQL-X system**

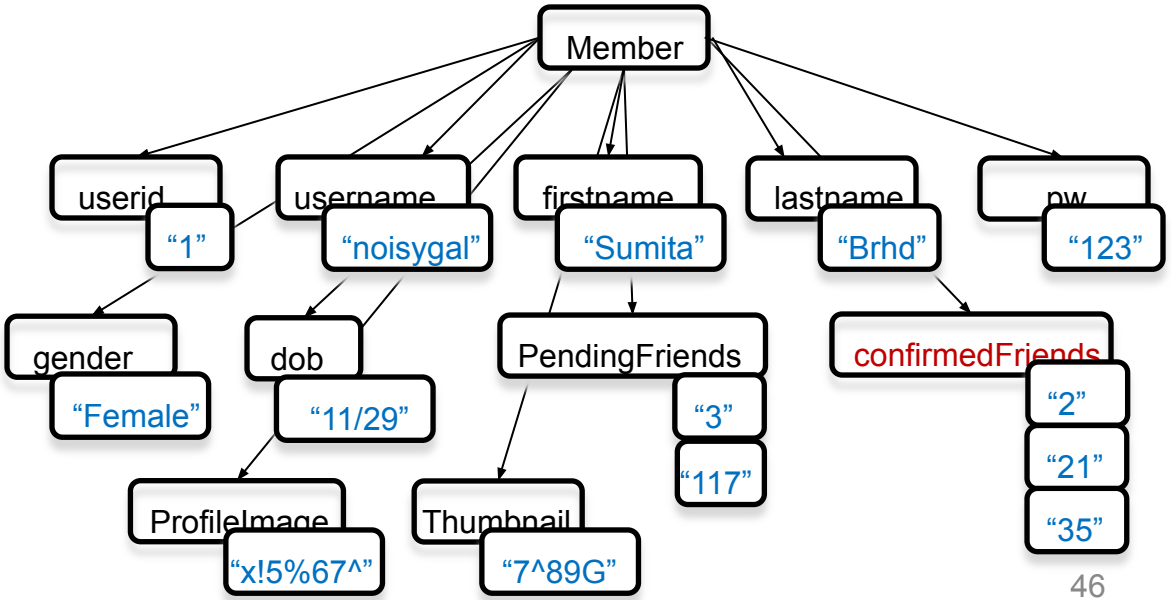
  Members(userid,username,pw,lastname,gender,dob,jdate,ldate,address,email, profileImage, thumbnail)

  Friends(userid1,userid2,status)

- **MongoDB , NoSQL, Document Store system – JSON like representation**

Members:{

"userid" : ""
"username": ""
"pw": ""
"firstname": ""
"lastname": ""
"gender" : ""
"dob" : ""
"pendingFriends":[]
"confirmedFriends":[]
"profileImage":[]
"thumbnail":[]
}



List Friends Action – Contd.

- **SQL-X system**

  Members(userid,username,pw,lastname,gender,dob,jdate,ldate,address,email, profileImage, thumbnail)

  Friends(userid1,userid2,status)

- **MongoDB , NoSQL, Document Store system – JSON like representation**

Members:{

"userid" : ""

"username": ""

"pw": ""

"gender" : ""

"dob" : ""

"pendingFriends":[]

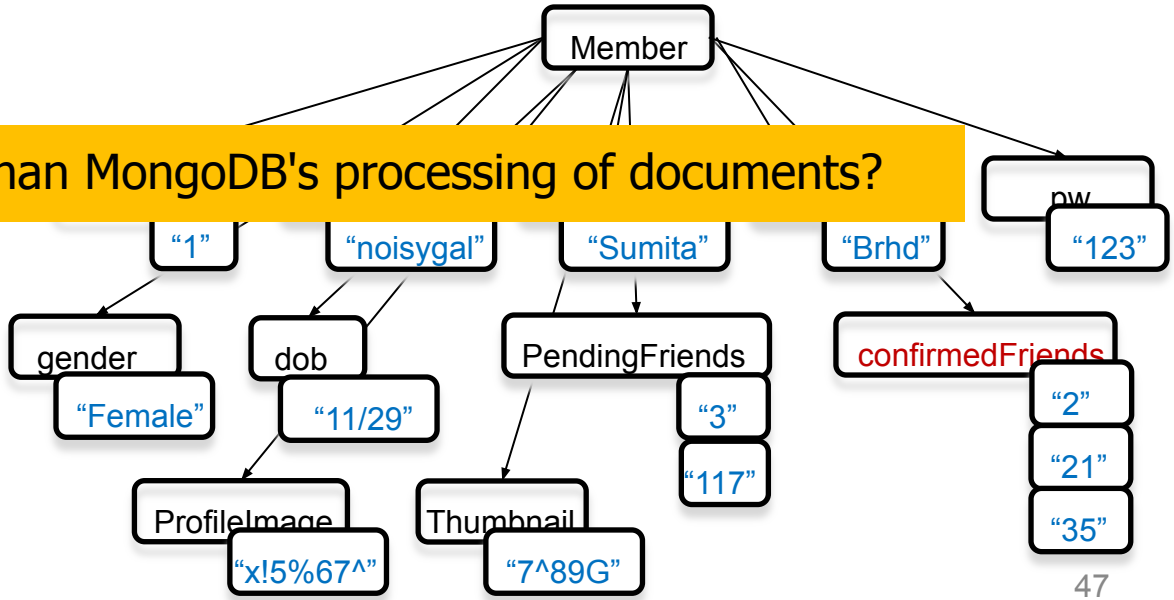
"confirmedFriends":[]

"profileImage":[]

"thumbnail":[]

}

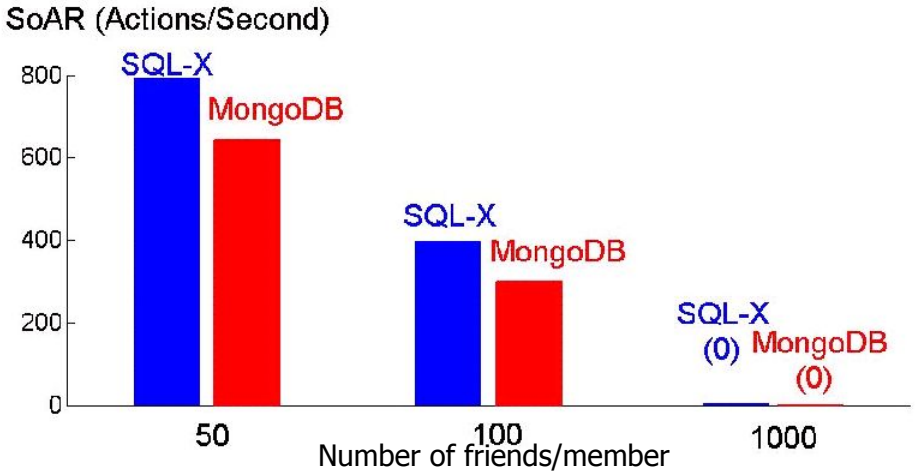
Is relational join slower than MongoDB's processing of documents?



Results – List Friends Action

USC Viterbi School of Engineering

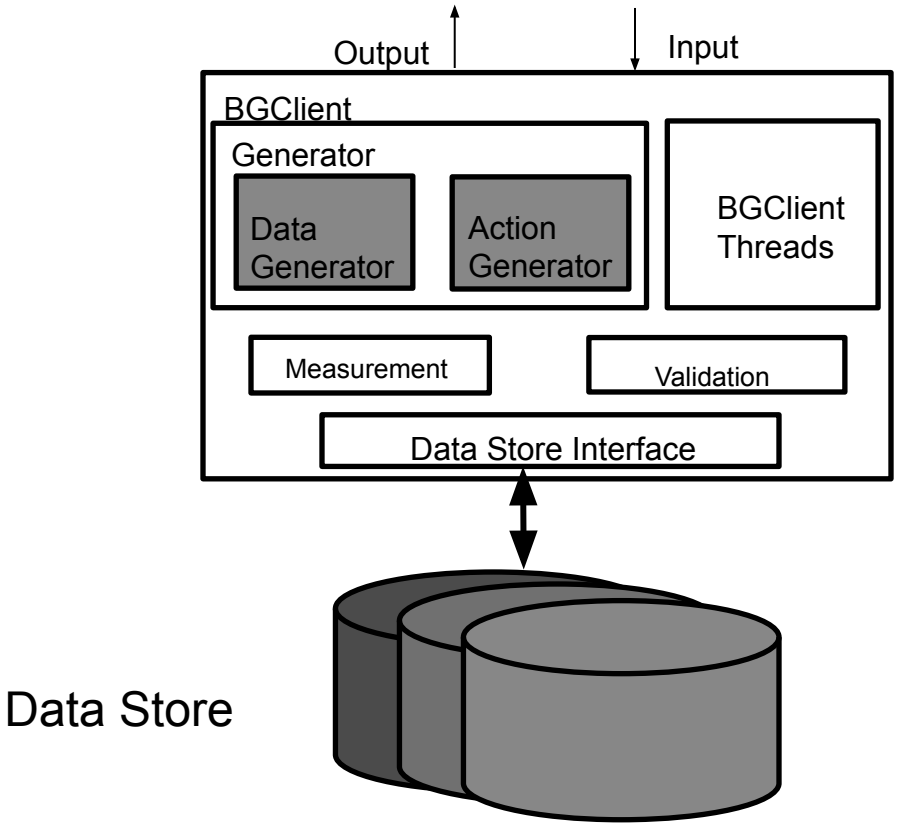
- List Friends Action



- Workload: 10k members, 12kB profile images and 2kB thumbnail images.
- With SQL-X the network becomes the bottleneck. With MongoDB the CPU becomes the bottleneck.
- SLA: 95% of actions to observe a response time below 100 msecs.

BG Software Architecture

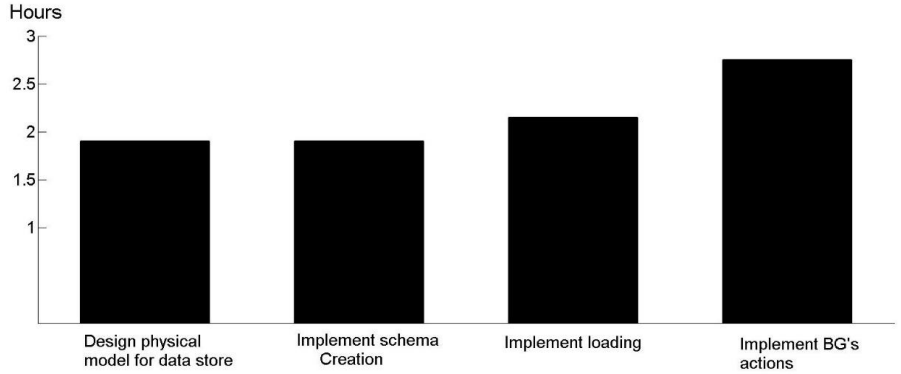
- BGClient Functions:
  - Schema Creation:
    - Creates database schema.
  - Load:
    - Populates database with data.
  - Benchmark:
    - Initiate a workload and measures performance.
- Modular



BG Software Architecture - Contd.

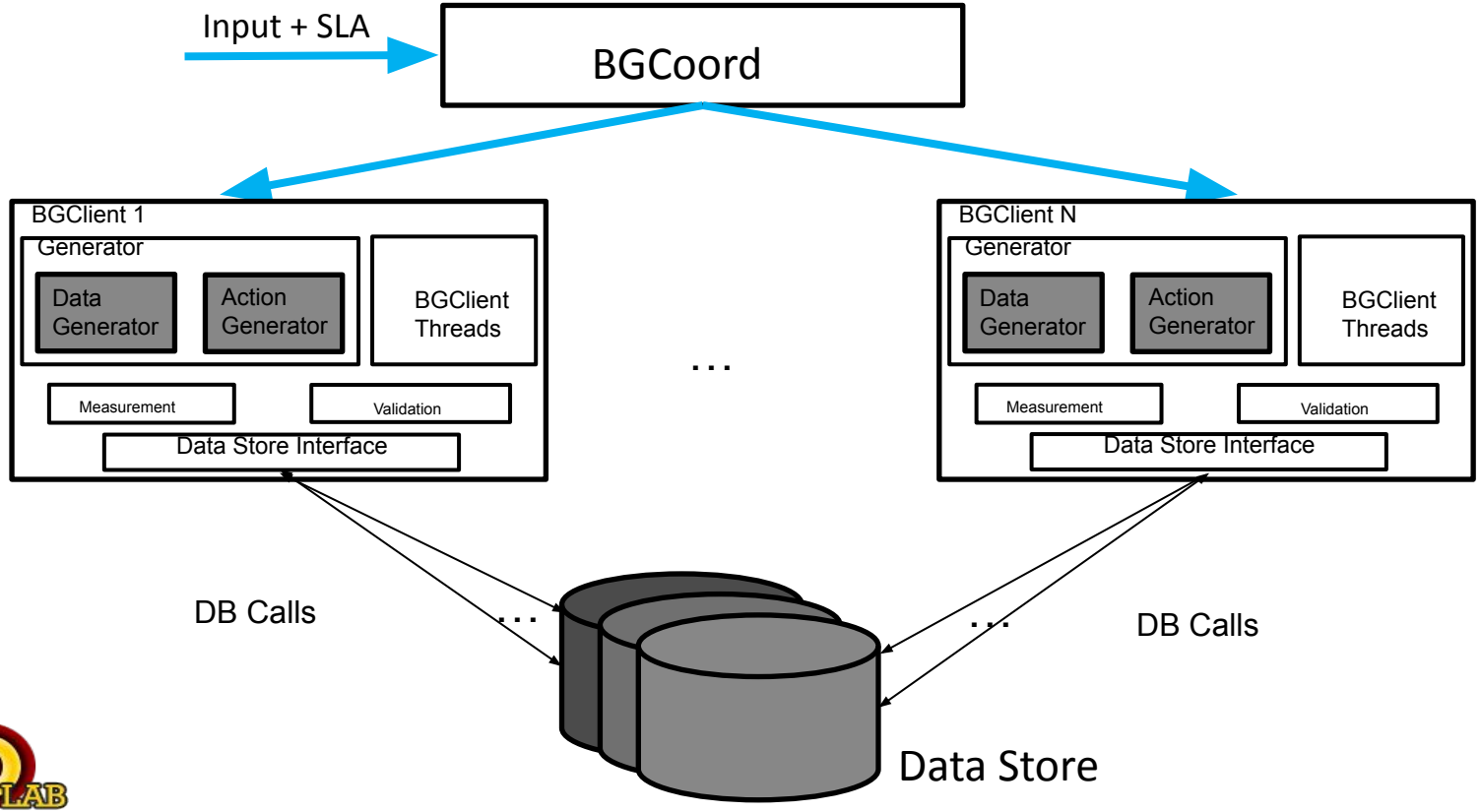
USC Viterbi School of Engineering

- Extensible
  - Extend to support new social actions
    - Retrieve mutual friends, search, suggest friends
      - ***Familiar user: 1 hour***
      - ***New comers (Students): 3-4 hours***
  - Extend to support new distributions
    - Poisson, Latest, Uniform
  - Extend to support new data stores
    - Surveys showed that competent users took about 7-10 hours to complete this.



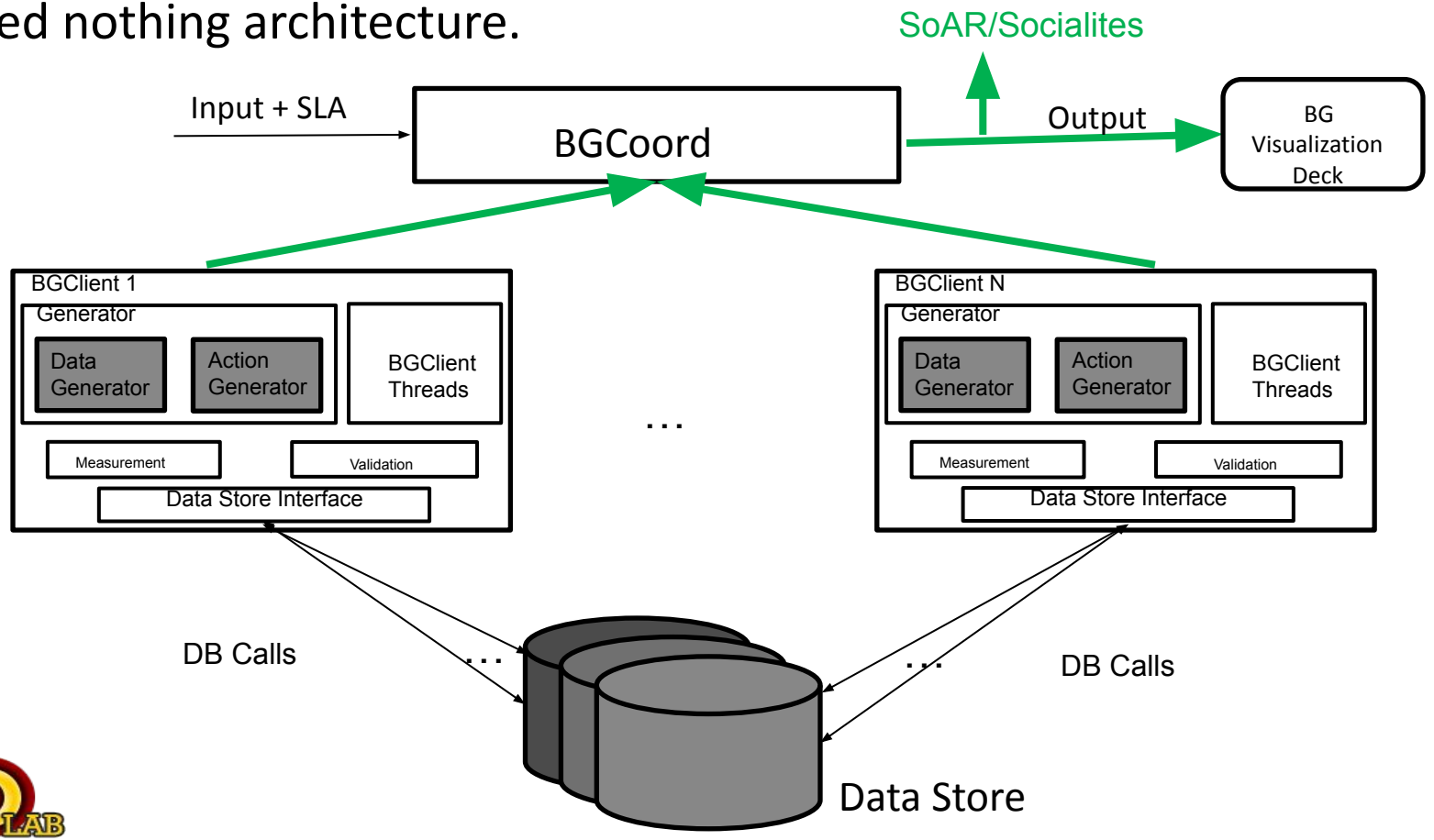
Scalability

- Shared nothing architecture.



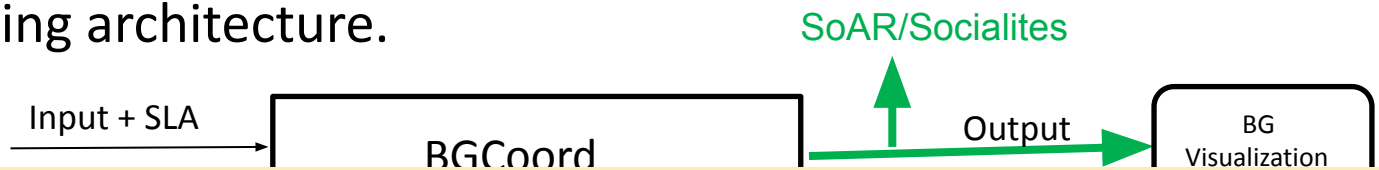
Scalability

- Shared nothing architecture.



Scalability

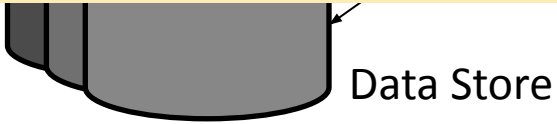
- Shared nothing architecture.



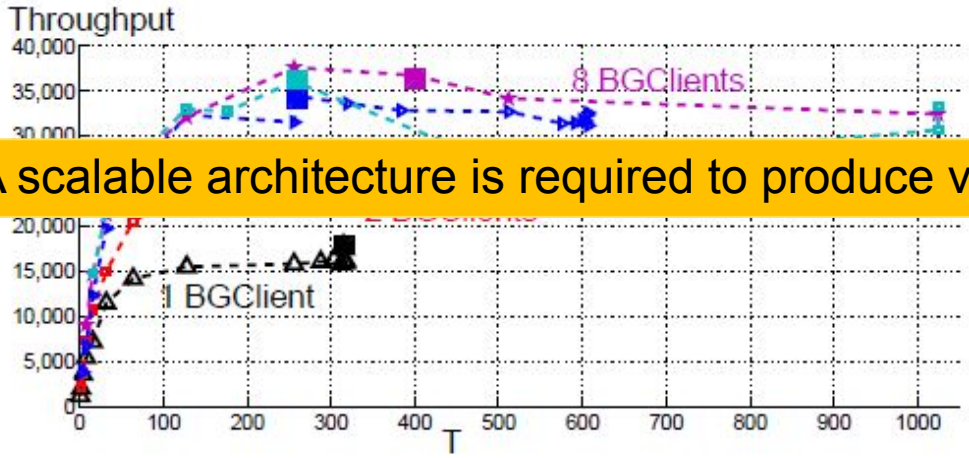
Requirement:
The BGClients should terminate at approximately the same time.

How?

1. A fraction of the emulated users is assigned to each BGClient.
2. A BGClient emulates a fixed number of threads representing concurrent socialites. This number is a function of the speed of its server relative to the other servers/BGClients. It is chosen such that all BGClients terminate at approximately the same time.
3. An emulated socialite issues requests serially.



Scalability – Contd.



A scalable architecture is required to produce valid ratings.

- Workload: View profile action with no images for members.
- With smaller number of BGClients the MongoDB client component is the bottleneck.
- With 8 and 16 BGClients the CPU of the DBMS server becomes the bottleneck.

Unpredictable Data: Validation Phase

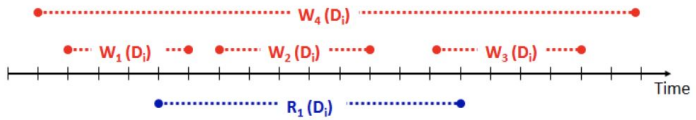
- Unpredictable data is either stale, inconsistent, or simply invalid data produced by a DBMS.
- BG is aware of the initial state of data. It generates actions. Hence, it knows the new state of the database.
- BG issues read and write actions to the DBMS. It knows the time stamp of the read and write actions, both when they are issued and when the DBMS completes processing them.
- There is a finite way for a read action to overlap the writes. BG enumerates these to compute the range of acceptable values that should be observed by a read action. If a data store produces a different value then it has produced unpredictable data. This is the validation phase of BG.

Unpredictable Data: Validation Phase

sources of a BGClient. During the benchmarking phase, each thread of a BGClient invokes an action that generates one log record. There are two types of log records, a read and a write log record corresponding to either a read or a write of a data item. A log record consists of a unique identifier, the action that produced it, the data item referenced by the action, its socialite session id, and start and end time stamp of the action. The read log record contains its observed value from the data store. The write log contains either the new value (named *Absolute Write Log*, AWL, records) or change (named *Delta Write Log*, DWL, records) to existing value of its referenced data item.

Validation Phase: How?

- When issuing requests, BG generates log records for read and write actions. They are written to the disk asynchronously.
- At the end of the experiment, BG:
  - Identifies the read actions with one or more overlapping write actions. There are 4 ways for a write to overlap a read operation.



  - BG computes the possible value that the read should observe. If the read observes a different value then the read has observed an unpredictable data.

Validation is an expensive operation. Hence, it is done in an offline manner, after an experiment completes.

Cache Augmented Architecture

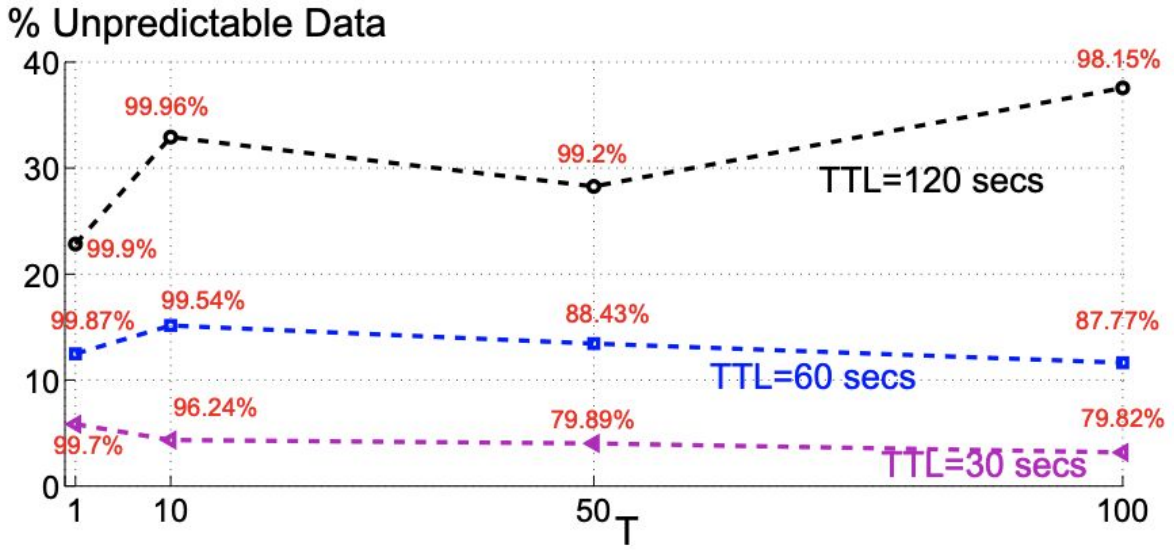
USC Viterbi School of Engineering

1. Value = Get (Key)
2. If Value is found, go to Step 6.
3. SQL queries
4. Query results
   - Application constructs Value using the results
5. Put(Key, Value)
6. Use Value to generate HTML result page

USC

What about writes?

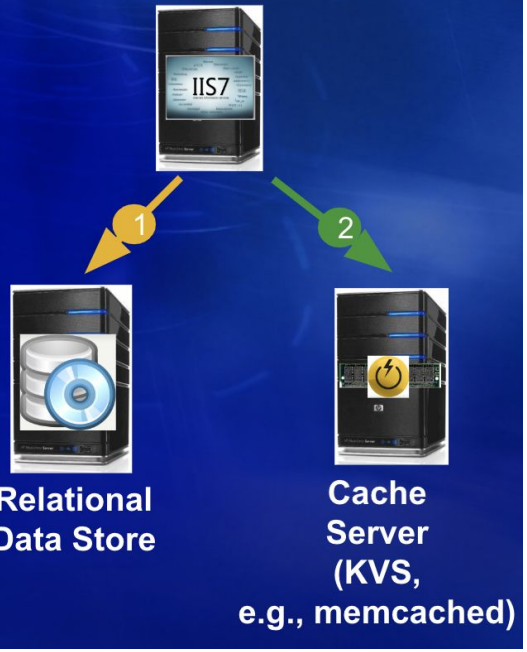
Set each cache entry to have a fixed time to live, say 60 seconds.



Amount of unpredictable data is independent of T.

Writes Invalidate Cache Entries

1. SQL DML Command: Insert, Delete, Update
2. Invalidate key-value pairs (Write-around)
   - Alternatives include Write-through (Refill/Refresh and incremental update) and Write-back.



A small amount of unpredictable data observed by 0.001% of reads due to race conditions. We will see solutions for this in Week 11.

BG Social Actions

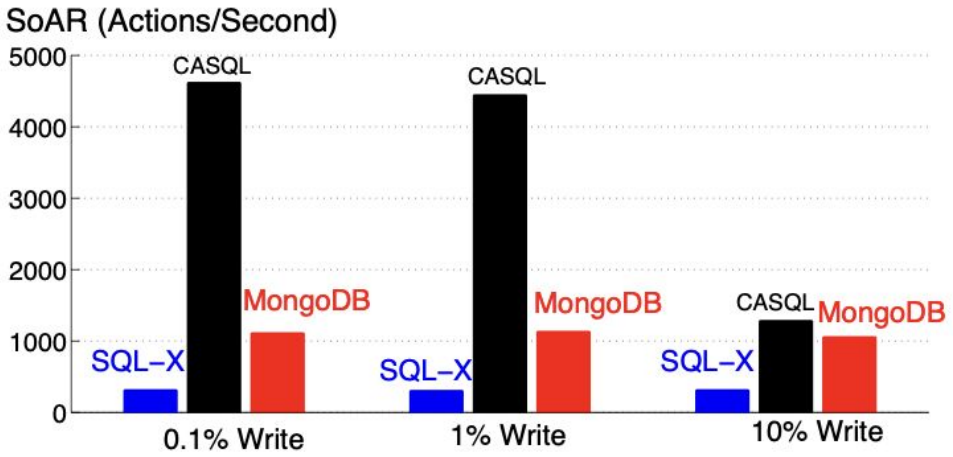
USC Viterbi School of Engineering

Action	facebook	g+		LinkedIn	YouTube
View Profile (VP)	✓	✓	✓	✓	✓
List Friends (LF)	✓	✓	✓	✓	✓
View Friend Requests (VFR)	✓	✘	✘	✓	✘
Invite Friend (IF)	✓	Add to Circle	Follow	✓	Subscribe
Accept Friend Request (AFR)	✓	✘	✘	✓	✘
Reject Friend Request (RFR)	✓	✘	✘	✓	✘
Thaw Friendship (TF)	✓	Remove from Circle	Unfollow	✓	Unsubscribe
View Top-K Resources (VTR)	✓	✓	✓	✓	✓
View Comments on a Resource (VCR)	✓	✓	✓	✓	✓
Post Comment on a Resource (PCR)	✓	✓	Reply to a tweet	Recommend a colleague's work	Post comment on a video
Delete Comment from a Resource(DCR)	✓	✓	Delete the reply for a tweet	Withdraw recommendation	Remove comment on a video
Share Resource (SR)	✓	✓	Post a tweet	Update profile	Upload a video
View News Feed (VNF)	✓	✓	✓	✓	✓

• S. Barahmand and S. Ghandeharizadeh. **BG: A Benchmark to Evaluate Interactive Social Networking Actions**. *CIDR '13, Asilomar, CA.*

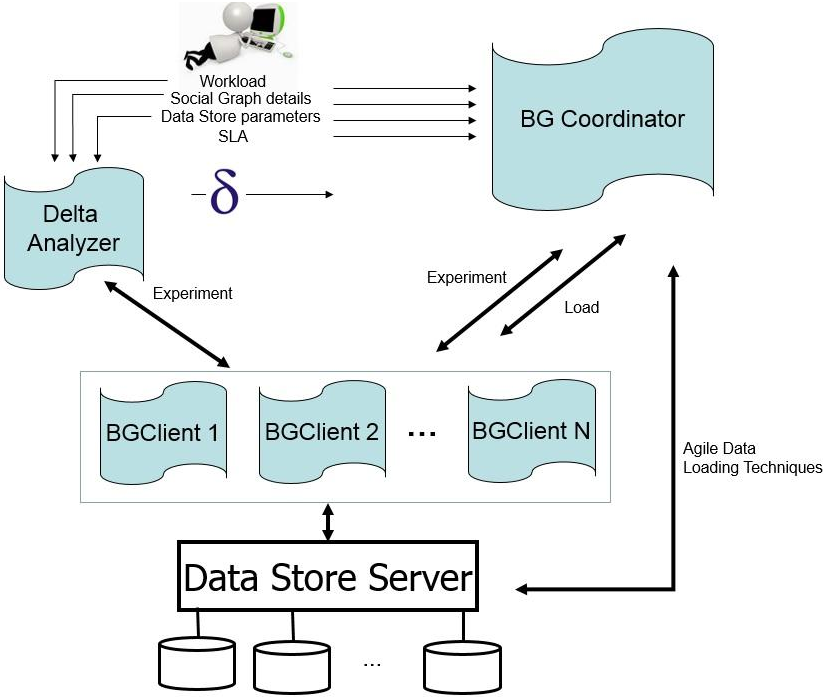
Mix of Read and Write Actions

BG Social Actions	Type	Very Low (0.1%) Write	Low (1%) Write	High (10%) Write
VP	Read	40%	40%	35%
LF	Read	5%	5%	5%
VFR	Read	5%	5%	5%
IF	Write	0.02%	0.2%	2%
AFR	Write	0.02%	0.2%	2%
RFR	Write	0.03%	0.3%	3%
TF	Write	0.03%	0.3%	3%
VTR	Read	49.9%	49%	45%
VCR	Read	0%	0%	0%
PCR	Write	0%	0%	0%
DCR	Write	0%	0%	0%

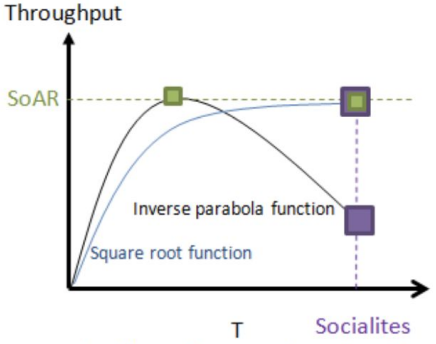


Quick Rating

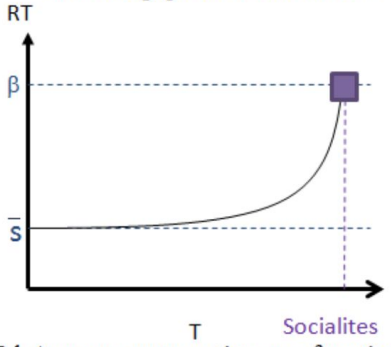
- Uses intelligent techniques to rate the system in a reasonable amount of time.



Assumptions:



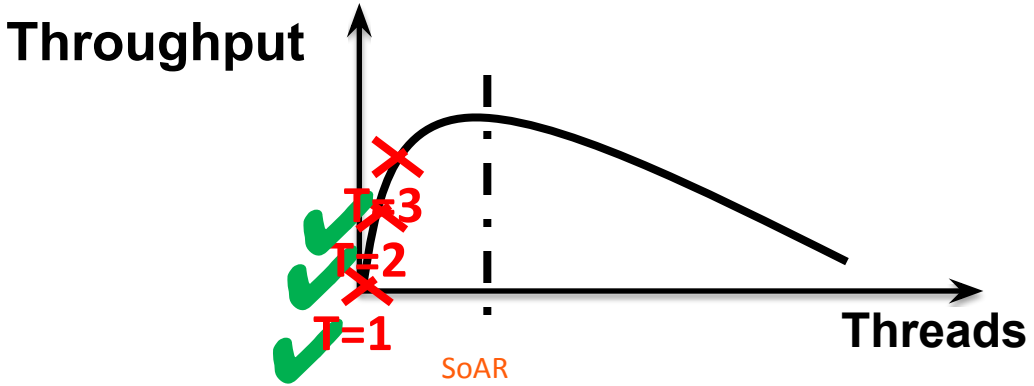
9.a Throughput as a function of T



9.b Average response time as a function of T

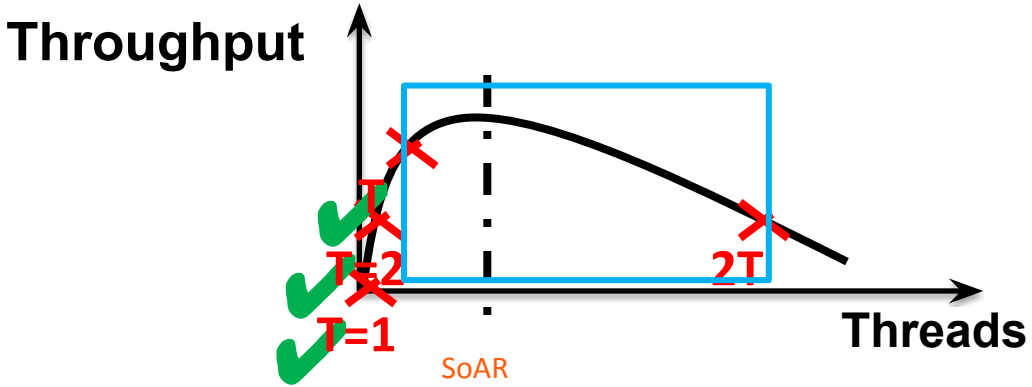
Quick Rating : Heuristic

- Number of experiments
  - A naive approach, increment the number of threads by one.



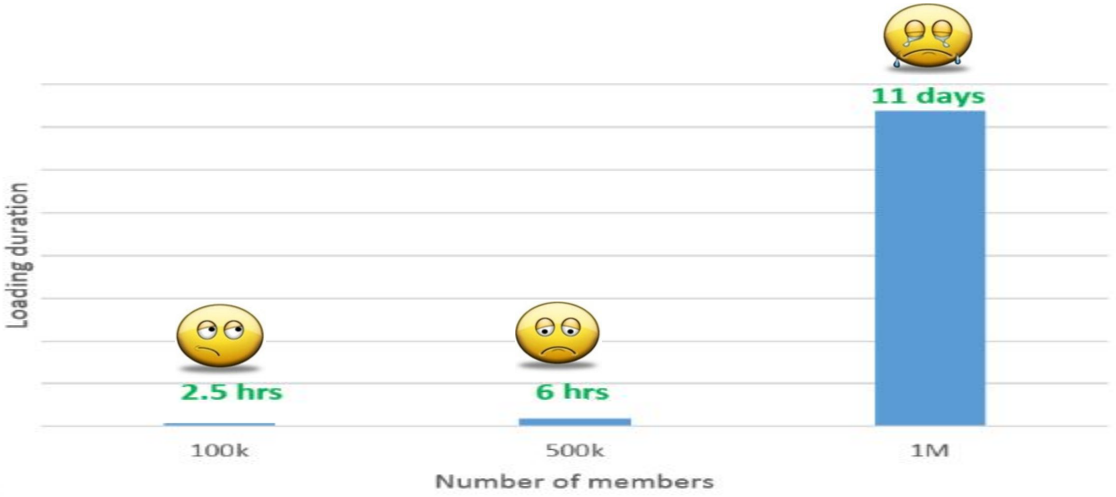
Quick Rating : Heuristic – Contd.

- Number of experiments
  - Heuristic search with +/- 10% margin of error which potentially reduces the number of conducted experiments.



Quick Rating : Two Agile Data Loading

- Loading the data store between the rounds of experiments with workloads that include writes:
  - Repeated construction and destruction of database is time consuming.



  - Agile data loading techniques to reduce the overhead of constructing the benchmark database.

Quick Rating : Two Agile Data Loading -Contd.

- Loading the data store between the rounds of experiments:
  - Database Image Loading (DBIL):
    - Relies on the capability of a data store to create a disk image of the database.
    - Uses this image repeatedly across different experiments.
    - Reduces 11 days to 30 minutes.
  - RepairDB:
    - Restores the database to its original state prior to the start of an experiment.
    - Reduces 11 days to 10 hours.

S. Barahmand and S. Ghandeharizadeh. **Expedited Rating of Data Stores Using Efficient Data Loading Techniques.** *CIKM '13, New York, NY.*

Quick Rating : Two Agile Data Loading -Contd.

- Loading the data store between the rounds of experiments:
  - Database Image Loading (DBIL):
    - Relies on the capability of a data store to create a disk image of the database.
    - Uses this image repeatedly across different experiments.
    - Reduces 11 days to 30 minutes.
  - RepairDB:
    - Restores the database to its original state prior to the start of an experiment.
    - Reduces 11 days to 10 hours.

S. Barahmand and S. Ghandeharizadeh. **Expedited Rating of Data Stores Using Efficient Data Loading Techniques.** *CIKM '13, New York, NY.*

BG Contributions

- Design, abstraction and implementation of a novel benchmark (BG) that can evaluate different classes of data stores used to manage social graphs.
  - Resembles a social networking system.
  - Generates meaningful requests.
  - Quantifies relevant metrics: Response time, Throughput, SoAR, Socialites, amount of unpredictable data.
  - Produces repeatable and explainable results.
  - Runs in a reasonable amount of time.
  - Used extensively to explore the behavior of alternative design decisions.
- Quantitative analysis and comparison of data stores used in social networks and demonstration of their tradeoffs.
  - Why systems behave as they do?
  - What are the tradeoffs between different systems/design decisions?

USC

USC Viterbi
School of Engineering

CSCI 585: EXAM 1

Exam 1:

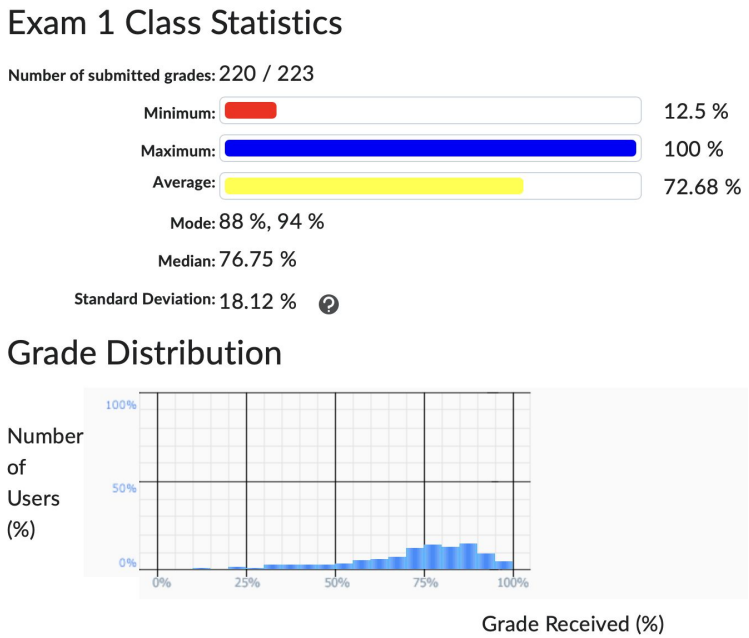
1. Grade distribution and class curve.
2. Introduction to graders.
3. Rubric used for grading.
4. Make the exam available to the students.

EXAM 1: Grades

Generous curve:

- 95-100 10 A+
- 90-100 20 A
- 80-90 57 A-/A
- 70-80 55 A-
- 60-70 27 B+/A-
- 50-60 18 B+
- 30-50 22 B/B+
- 12-30 6 B

- Minimum: 12.5
- Maximum: 100
- Average: 72.68
- Median (50th Percentile): 76.5

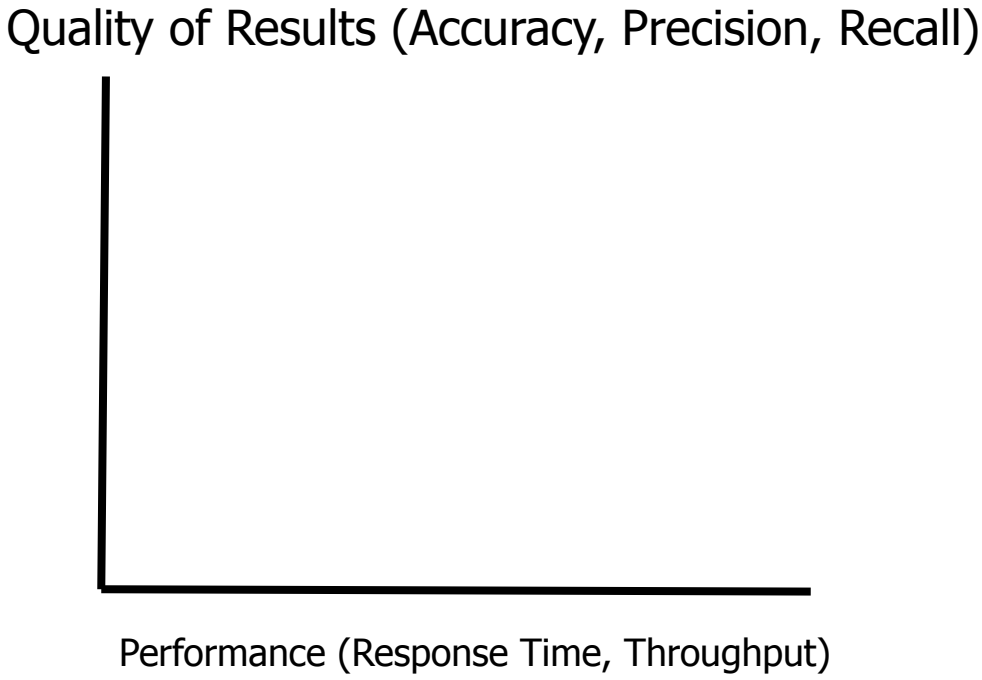


USC

USC Viterbi
School of Engineering

CSCI 585: Discussion

Big Picture

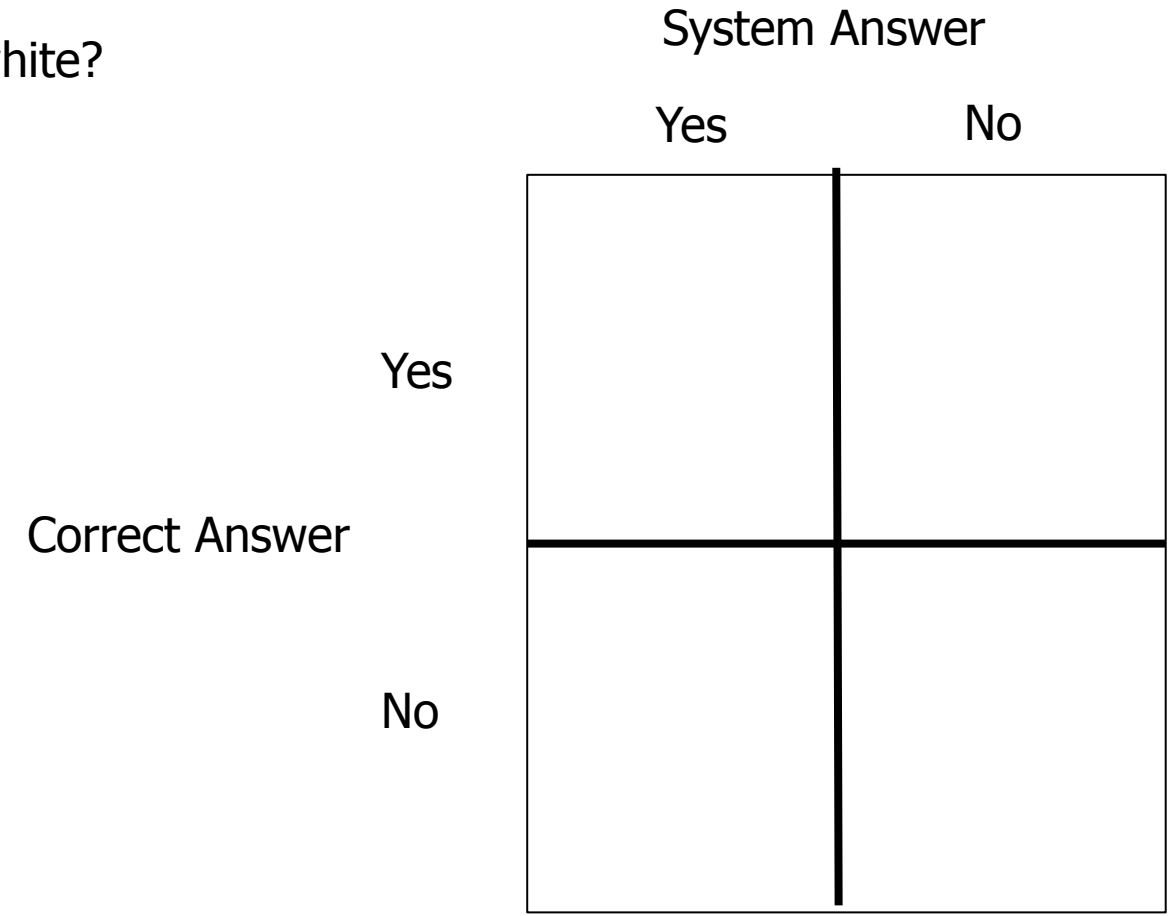


How about quality of results (unpredictable data)?

Truth Matrix: Binary Responses

USC Viterbi School of Engineering

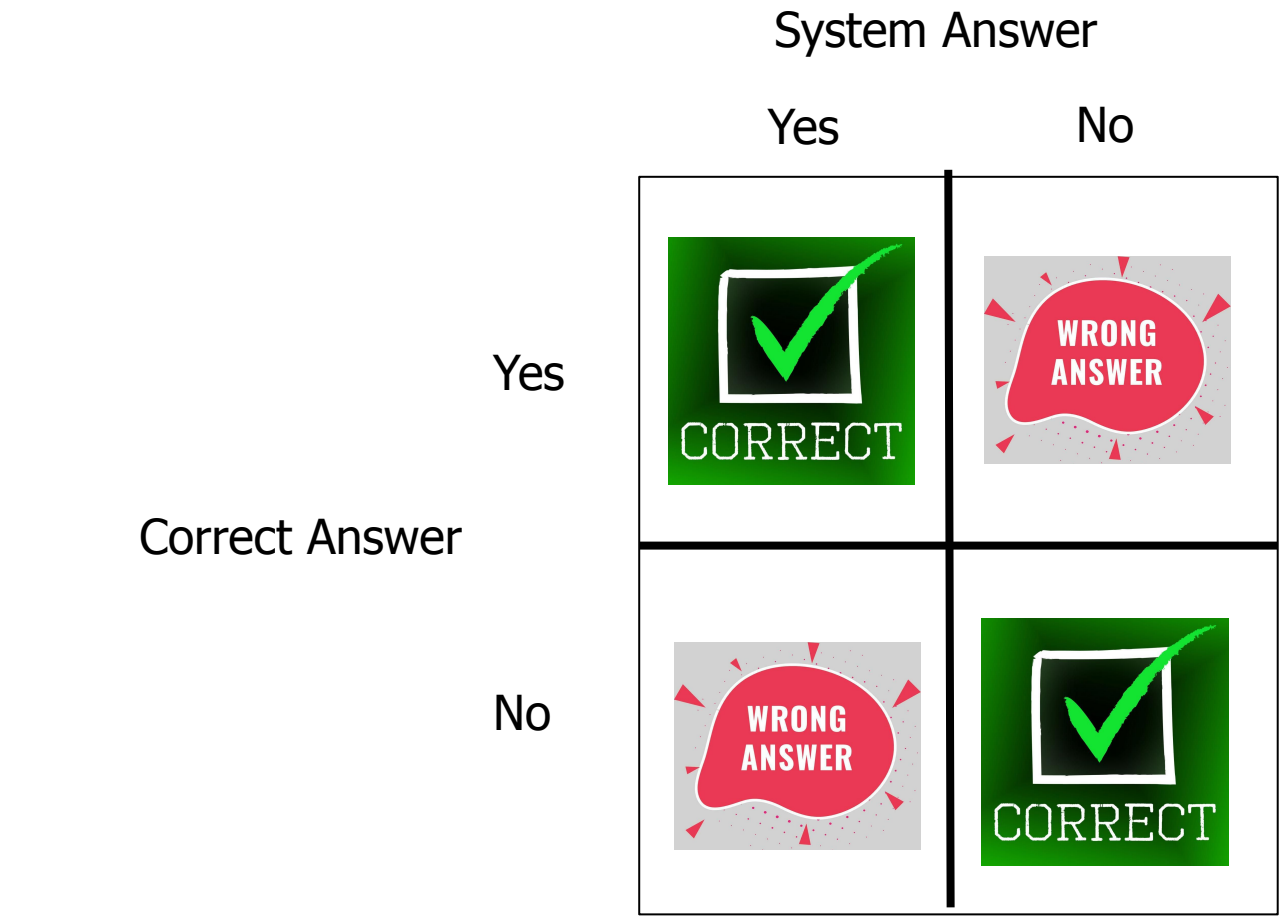
Is the color of this shirt white?



USC

Truth Matrix: Correct & Wrong

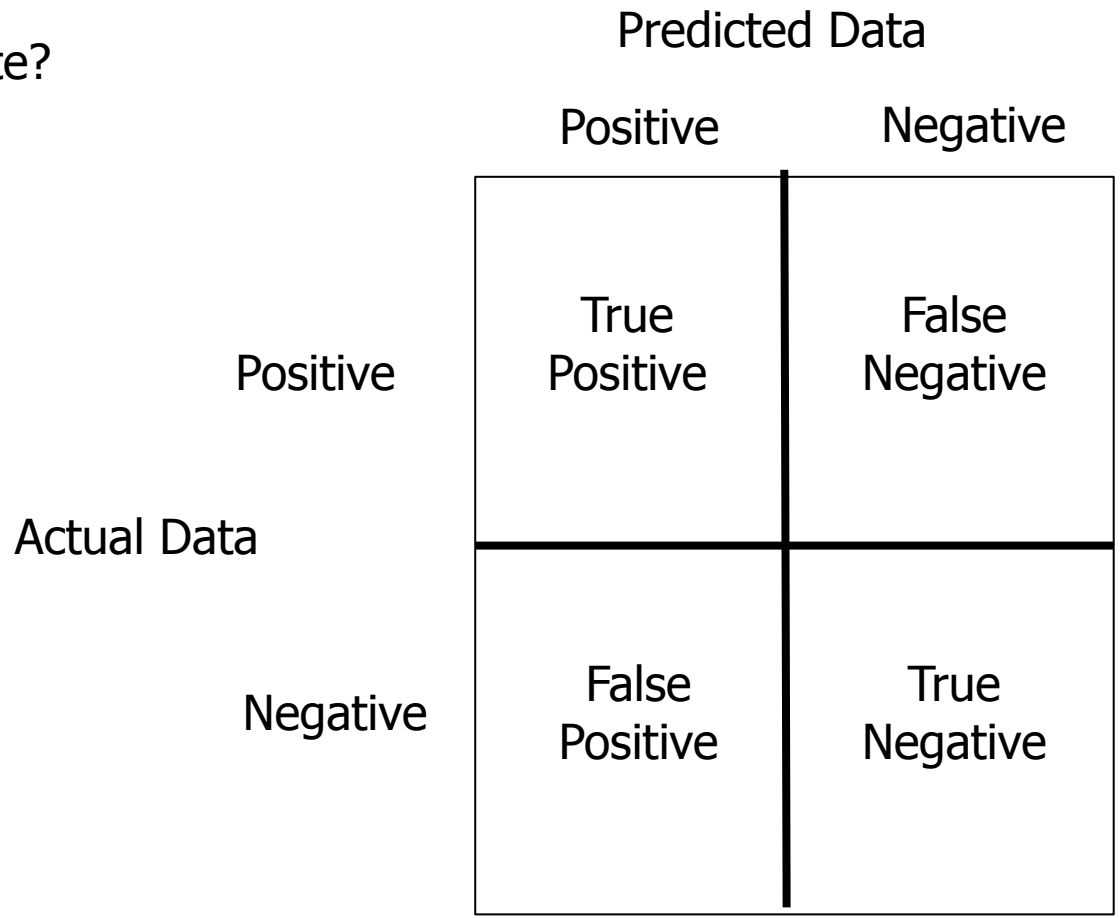
Does this plane fly?



Confusion Matrix

USC Viterbi School of Engineering

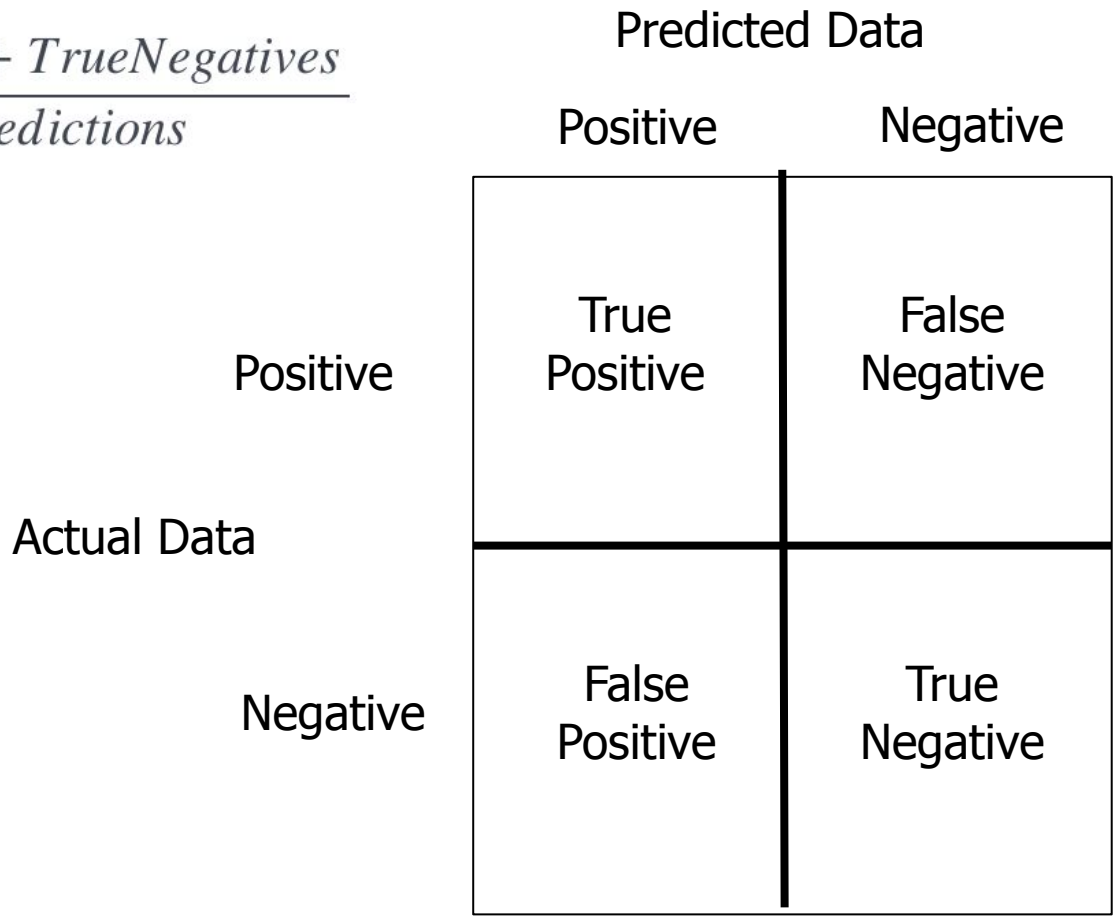
Is the color of this shirt white?



USC

Confusion Matrix: Accuracy

$$Accuracy = \frac{TruePositives + TrueNegatives}{TotalPredictions}$$



Accuracy: Limitation

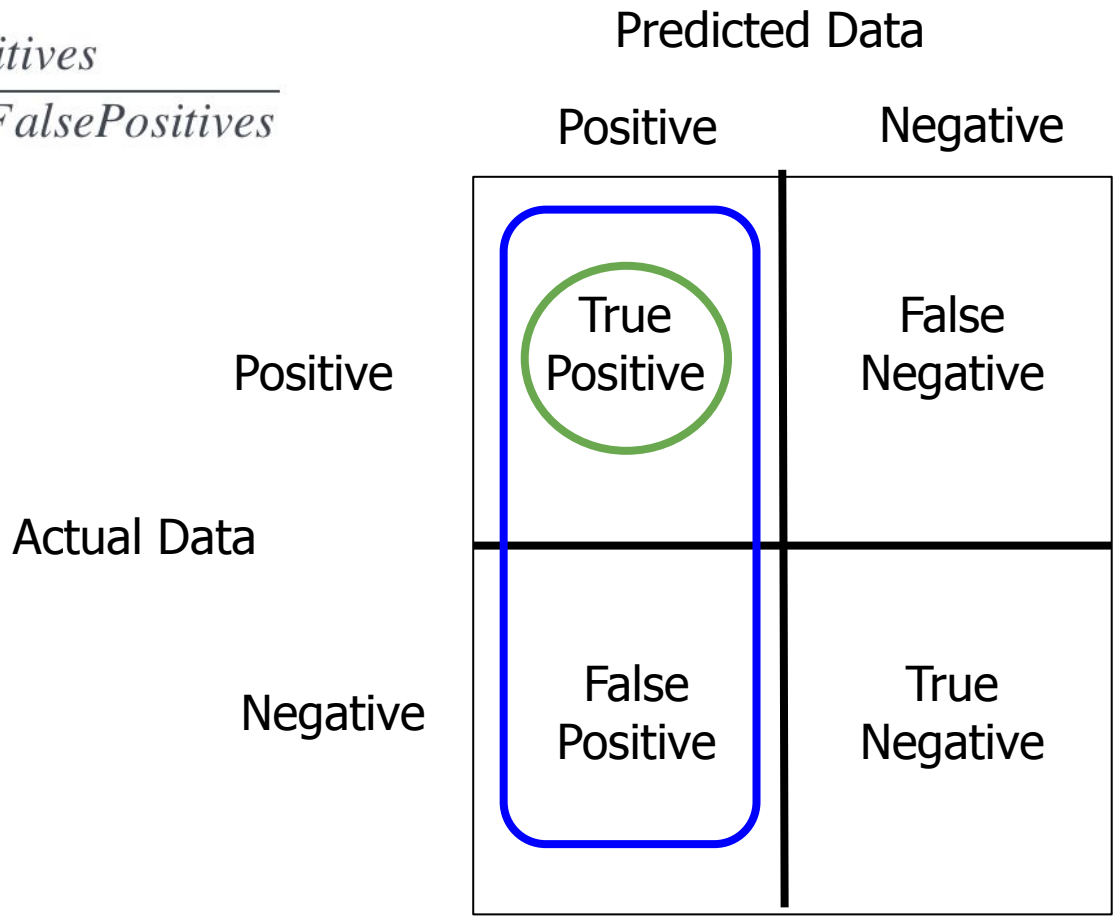
- Accuracy may be misleading in certain scenarios.
  - Example: With a pool of 10,000 patients, 10 are sick. If the system predicts that all patients are healthy then its accuracy is 99.9%, i.e., 9,990/10,000.

$$Accuracy = \frac{TruePositives + TrueNegatives}{TotalPredictions}$$

In our example, what are the values of TruePositivies, TrueNegatives, and TotalPredictions?

Confusion Matrix: Precision

$$Precision = \frac{TruePositives}{TruePositives + FalsePositives}$$



Precision: Example

- Example: With a pool of 10,000 patients, 10 are sick. If the system predicts that all patients are healthy then its accuracy is 99.9%, i.e., 9,990/10,000.
- Precision is 0.

$$Precision = \frac{TruePositives}{TruePositives + FalsePositives}$$

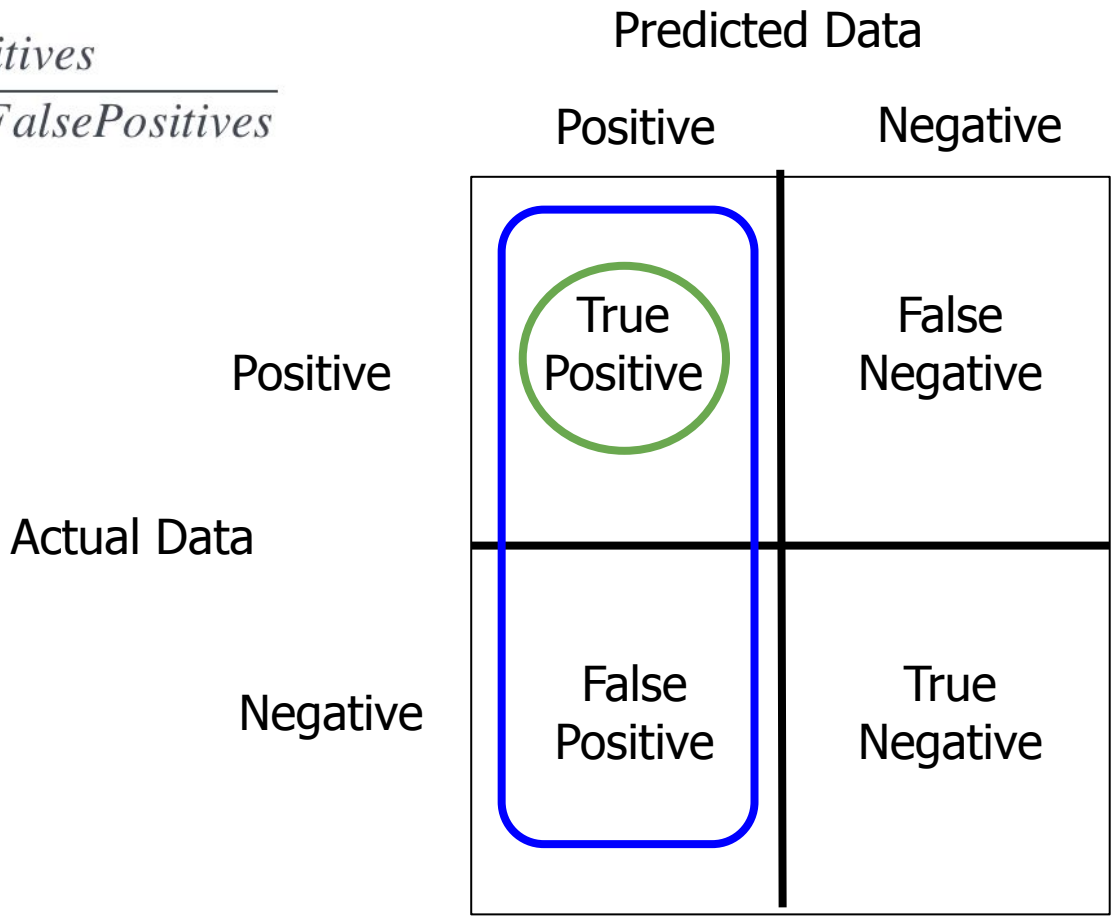
In our example, what are the values of TruePositivies and FalsePositives?

Precision: Meaning

$$Precision = \frac{TruePositives}{TruePositives + FalsePositives}$$

Precision tells you how correct the system's positive response are.

Out of everything the system said were positive. What percentage was actually positive?



Precision: Limitation

- Example: With a pool of 10,000 transactions, 100 are fraudulent. If the system predicts 1 fraudulent transaction as fraud then its precision is 100%, 1/1

$$Precision = \frac{TruePositives}{TruePositives + FalsePositives}$$

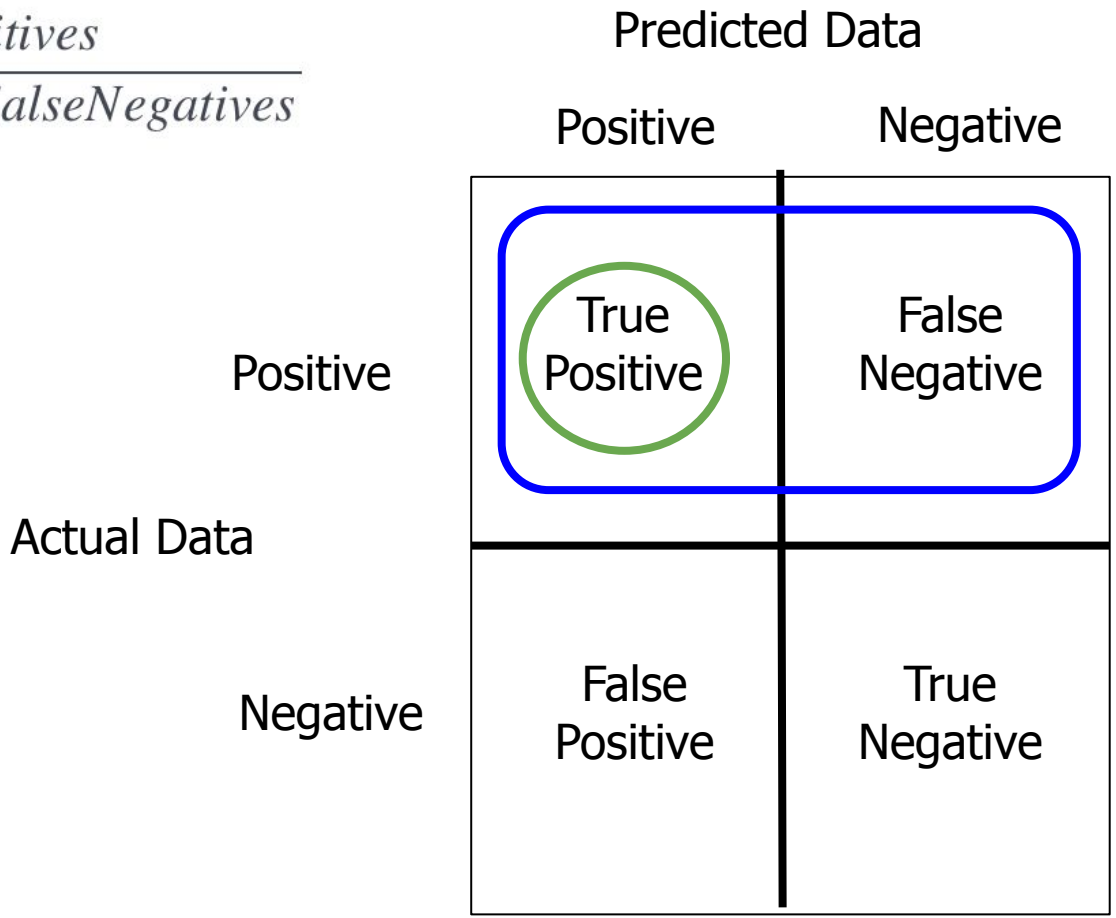
99% of fraudulent transactions are missed causing the company to go bankrupt.

Confusion Matrix: Recall

$$Recall = \frac{TruePositives}{TruePositives + FalseNegatives}$$

Recall tells you completeness of positive results.

Out of all the responses that were *actually* positive, what percentage did the system identify correctly?



Recall: Example

- Example: With a pool of 10,000 transactions, 100 are fraudulent. If the system predicts 1 fraudulent transaction as fraud then its precision is 100%
- Recall is 1%.

$$Recall = \frac{TruePositives}{TruePositives + FalseNegatives}$$